



Projet de physique des télécommunications : FDTD

Auteurs :

Louis Bertrand

Laurent Béguelin

Professeur :

DE DONCKER Philippe

Table des matières

1	FDTD 1D	1
1.1	Paramètres physiques	1
1.2	Pertes électriques et diélectriques	2
1.3	Critère de stabilité	3
1.4	Paramètres temporels	3
1.5	Optimisation MATLAB	4
1.6	Conditions aux limites	5
1.7	Sources	6
1.8	Affichage Matlab	7
	Résultat et simulation 1D	8
2	FDTD 2D	11
2.1	Paramètres physiques	11
2.2	Dimensionnement de la grille	11
2.3	Paramètres de temps	12
2.4	Équations de Maxwell en 2D	12
	Vecteurs courbes d'évolution des champs	14
2.5	Mise à jour du champ E_z	15
2.6	Mise à jour des champs $\tilde{B}_x$ et $\tilde{B}_y$	16
2.7	Conditions aux bords	17
2.8	Perfect Match Layer (PML)	19
2.9	Sources	21
2.10	Obstacles	22
2.11	Puissance et tension captées par une antenne de hauteur équivalente et résistance connues	24
2.12	Affichage et enregistrement Matlab	25
	Résultat et simulation 2D	27
	Propagation d'une onde sinusoïdale dans le vide	27

TABLE DES MATIÈRES

	Propagation d'une onde sinusoïdale à travers un diélectrique	28
	Puissance et tension réceptionnées par une antenne dipôle $\lambda/2$	28
	Conditions de Dirichlet en 2D (sans PML)	30
2.13	Réflexion/Transmission à travers un mur	34
2.14	Propagation guidée	36

Introduction

Dans le cadre du cours de physique des télécommunications dispensé par le professeur Philippe De Doncker à l'Université libre de Bruxelles (ULB), les étudiants sont invités à implémenter la méthode de Yee afin de résoudre les équations de Maxwell et de simuler le comportement des ondes électromagnétiques. Cette approche est d'abord appliquée en une dimension, puis étendue en deux dimensions, avec l'introduction de milieux diélectriques pour étudier l'effet des différentes matières sur la propagation des ondes.

La première partie est consacrée aux simulations 1D, la deuxième aux simulations 2D, et la dernière présente des simulations sélectionnées pour illustrer les chapitres 7, 8 et 9 du cours.

1 FDTD 1D

Pour implémenter la méthode de Yee en 1D, il suffit de créer (puis modifier grâce aux formules discrètes dont nous allons parler par la suite) des vecteurs de taille N_x (nombre de points constituant E et B dans le plot Matlab) donnant la valeur de B et E à un temps donné (indice q) en fonction de leur position spatiale (indice m , de taille N_x). Puis, dans une boucle temporelle avec un nombre suffisant d'itérations pour pouvoir observer les différents phénomènes ciblés, nous allons modifier E en fonction de B (et inversement) selon la méthode vue dans l'énoncé du projet. De plus on considère $\tilde{B} = cB_y$. Pour rappel (voir consignes du projet) :

$$\mathbf{E}_z^{q+1}[m] = \mathbf{E}_z^q[m] + \frac{1}{\varepsilon\mu c\Delta x} \Delta t \left(\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2] - \tilde{\mathbf{B}}_y^{q+1/2}[m-1/2] \right) - \frac{\Delta t}{\varepsilon} J_z^{q+1/2}[m] \quad (1)$$

$$\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2] = \tilde{\mathbf{B}}_y^{q-1/2}[m+1/2] + \frac{c\Delta t}{\Delta x} (\mathbf{E}_z^q[m+1] - \mathbf{E}_z^q[m]) \quad (2)$$

Remarque : À la différence de l'équation utilisée dans les consignes du projet, nous avons pris les équations de Maxwell dans un milieu quelconque, c'est-à-dire que ce ne sont plus la permittivité et perméabilité du vide (ε_0 et μ_0) mais bien celles d'un milieu quelconque (ε et μ) (Voir cours [2]).

1.1 Paramètres physiques

Les paramètres physiques du code en 1D sont implémentés de la sorte :

```

1  % Paramètres du vide :
2  c0 = 3e8;           % Vitesse de la lumière [m/s]
3  e0 = 8.8e-12;       % Permittivité du vide [F/m]
4  u0 = 4*pi*1e-7;     % Perméabilité du vide [H/m]
5
6  % Paramètres du matériau :
7  er = 1;             % Permittivité relative du matériau
8  ur = 1;             % Perméabilité relative du matériau
9  sigma_e = 5e-3;     % Conductivité électrique du matériau [S/m]
10
11 % Fréquence [Hz] :
12 fmax = 1e9;

```

1.2 Pertes électriques et diélectriques

Afin de faire apparaître la conductivité σ dans l'équation 1, on utilise la loi d'Ohm locale : $\vec{\mathbf{J}} = \sigma \vec{\mathbf{E}}$ (voir cours d'électricité BA2 [9]). Dans un matériau réel, σ regroupe la conductivité électrique statique σ_{stat} , associée aux pertes ohmiques par conduction libre (effet Joule), et une conductivité équivalente $\sigma_{\text{eq}} = \omega \varepsilon''$ représentant les pertes diélectriques dues à la dissipation interne lors du déplacement amorti des charges liées. On obtient ainsi $\sigma_{\text{total}} = \sigma_{\text{stat}} + \omega \varepsilon''$. Dans l'équation 1, $J_z^{q+\frac{1}{2}}$ est défini à l'indice temporel $q + \frac{1}{2}$, tandis que $\mathbf{E}_z$ est connu aux instants q et $q + 1$. Pour lever ce décalage, on adopte l'approximation valable pour un pas de temps suffisamment petit (voir cours d'analyse numérique BA2 [1]).

$$J_z^{q+\frac{1}{2}} = \sigma \mathbf{E}_z^{q+\frac{1}{2}} \approx \sigma \cdot \frac{\mathbf{E}_z^{q+1}[m] + \mathbf{E}_z^q[m]}{2} \quad (3)$$

En injectant cette approximation dans l'équation 1, on a :

$$\varepsilon \cdot \frac{\mathbf{E}_z^{q+1}[m] - \mathbf{E}_z^q[m]}{\Delta t} + \sigma \cdot \frac{\mathbf{E}_z^{q+1}[m] + \mathbf{E}_z^q[m]}{2} = \frac{1}{\mu c \Delta x} \cdot \tilde{\mathbf{B}}_y^{q+1/2}[m+1/2] - \tilde{\mathbf{B}}_y^{q+1/2}[m-1/2] \quad (4)$$

L'équation de mise à jour de $\mathbf{E}_z$ en fonction de $\mathbf{B}_y$ devient donc :

$$\mathbf{E}_z^{q+1}[m] = \left(\frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}} \right) \cdot \mathbf{E}_z^q[m] + \left(\frac{\Delta t}{\mu c \varepsilon (1 + \frac{\sigma \Delta t}{2\varepsilon})} \right) \cdot \frac{\left(\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2] - \tilde{\mathbf{B}}_y^{q+1/2}[m-1/2] \right)}{\Delta x} \quad (5)$$

Remarque : Dans les consignes du projet, il a été proposé de considérer le champ électrique constant entre deux demi-itérations, ce qui revient à dire que $J_z^{q+\frac{1}{2}} \approx J_z^q = \sigma \mathbf{E}_z^q$. Or cela reste légèrement moins correct que l'approximation faite ici, d'où notre choix de garder ces coefficients

pour les calculs avec pertes.

1.3 Critère de stabilité

Comme montré dans les consignes du projet (et de manière plus approfondie ici [7]), nous avons des contraintes sur le pas de temps et d'espace pour assurer une implémentation stable de la FDTD 1D. Dans le code, elles sont définies comme tel :

```

1  % fréquence maximale :
2  fmax = 1e9;
3
4  % Longueur d'onde minimale :
5  lambda_min = c0/(fmax);
6
7  % définitions spatiale et temporelle :
8  N = 20 ;           % Car la bonne pratique impose N > 6 mais pas plus grand que 20 pour limiter la charge de
   ↪ calcul
9  dx = lambda_min/N; % Pour vérifier la stabilité du système
10 dt = dx/(2*c0);   % En découle le pas temporel par bonne pratique

```

1.4 Paramètres temporels

Pour déterminer la taille de la grille temporelle minimum requise (Step), nous allons calculer le temps de propagation (t_{prop}) que prends une onde pour se propager dans la grille spatiale. ce dernier est trouvé de cette manière (Source [7]) :

$$t_{prop} = \frac{N_x \cdot \Delta x}{v} \quad \text{où } v \text{ est la vitesse de phase de l'onde} \quad (6)$$

En bonne pratique, on prendra le temps total T_{tot} et donc un nombre total d'itération $STEPS$ (Source : [7]) égale à :

$$T_{tot} = 12 \cdot \tau + 5 \cdot t_{prop} \quad \text{où } \tau \text{ est la taille de l'impulsion du signal de source (eq 13)} \quad (7)$$

$$STEPS = \frac{T_{tot}}{\Delta t} \quad (8)$$

Cela donne :

```

1 % Temps de la simulation totale :
2 t_prop = (Nx*dx)/c0;           % Temps de propagation d'un coté a l'autre de la grille
3 T_tot = 12*tau + 5*t_prop;      % Formule de bonne pratique
4 Steps = ceil(T_tot/dt);         % Nombre d'itération temporelle

```

1.5 Optimisation MATLAB

Comme première optimisation, nous allons utiliser les relations $c^2 = \frac{1}{\epsilon_0 \mu_0}$ ainsi que $\epsilon = \epsilon_0 \cdot \epsilon_r$ et $\mu = \mu_0 \cdot \mu_r$ afin de limiter le nombre de valeurs flottantes (à virgule, aussi appelées 'float') à calculer et donc améliorer la précision de la valeur numérique des champs en chaque point (car Matlab approxime les calculs en float et le cumul de ces approximations peut amener à des erreurs notables, voir cours d'analyse num. [1]). L'équation 5 devient donc :

$$\mathbf{E}_z^{q+1}[m] = \left(\frac{1 - \frac{\sigma \Delta t}{2\epsilon}}{1 + \frac{\sigma \Delta t}{2\epsilon}} \right) \cdot \mathbf{E}_z^q[m] + \left(\frac{c \Delta t}{\mu_r \epsilon_r (1 + \frac{\sigma \Delta t}{2\epsilon})} \right) \cdot \frac{\left(\tilde{\mathbf{B}}_y^{q+1/2}[m + 1/2] - \tilde{\mathbf{B}}_y^{q+1/2}[m - 1/2] \right)}{\Delta x} \quad (9)$$

Ensuite, pour éviter de calculer les coefficients des équations 9 et 2 à chaque itération temporelle (et donc augmenter le temps de calcul, ce qui revient à une perte de 'Frame Per Seconde' (FPS) notable des simulations), nous allons les calculer en dehors de la boucle en posant les coefficients m_{B_y} , $m_{E_z,1}$ et $m_{E_z,2}$ tels que :

— **Coefficient de pertes électriques :**

$$m_{E_z,1} = \frac{1 - \frac{\sigma \Delta t}{2\epsilon}}{1 + \frac{\sigma \Delta t}{2\epsilon}}$$

— **Coefficient lié à la variation spatiale de $\mathbf{B}_y$ dans la mise à jour de $\mathbf{E}_z$:**

$$m_{E_z,2} = \frac{\Delta t}{\epsilon \left(1 + \frac{\sigma \Delta t}{2\epsilon} \right)}$$

— **Coefficient lié à la variation spatiale de $\mathbf{E}_z$ dans la mise à jour de $\tilde{\mathbf{B}}_y$:**

$$m_{B_y} = c \cdot \Delta t$$

Une fois implémenté sur Matlab :

```

1  % Calcul des coefficients (avec perte) :
2  mEz1 = (1 - SIGMA_E*dt./(2*e0*ER))./(1 + SIGMA_E*dt./(2*e0*ER));
3  mEz2 = c0*dt./(ER.*UR.*(1 + SIGMA_E*dt./(2*e0*ER)));
4  mBy = c0*dt;

```

Les équations de mise à jour de $\mathbf{E}_z$ et de $\tilde{\mathbf{B}}_y$ deviennent donc :

$$\mathbf{E}_z^{q+1}[m] = m_{Ez,1} \cdot \mathbf{E}_z^q[m] + m_{Ez,2} \cdot \frac{\left(\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2] - \tilde{\mathbf{B}}_y^{q+1/2}[m-1/2] \right)}{\Delta x} \quad (10)$$

$$\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2] = \tilde{\mathbf{B}}_y^{q-1/2}[m+1/2] + m_{By} \cdot \frac{\left(\mathbf{E}_z^q[m+1] - \mathbf{E}_z^q[m] \right)}{\Delta x} \quad (11)$$

Les mises à jour (dans la boucle principale) sur Matlab donnent :

```

1  %% Boucle principale
2  for T = 1 : Steps
3      % Mise à jour de B par E :
4      for nx = 1:Nx-1
5          By(nx) = By(nx) - mBy*(Ez(nx+1) - Ez(nx))/dx;
6      end
7      ...
8      % Mise à jour de E par B :
9      for nx = 2:Nx
10         Ez(nx) = mEz1(nx)*Ez(nx) - mEz2(nx)*(By(nx) - By(nx-1))/dx;
11     end
12     ...
13 end

```

1.6 Conditions aux limites

Les équations 10 et 11 ne peuvent pas être appliquées aux bords du domaine, en $m = 1$ et $m = M$, car elle nécessite des points voisins hors grille. Pour remédier à cela, des conditions aux limites sont obligatoires. Nous allons en proposer 2 :

Les conditions absorbantes (voir consignes de projet) :

$$E_z^q[1] = E_z^{q-2}[2], \quad E_z^q[M] = E_z^{q-2}[M-1]$$

Pour ce qui est des conditions à gauche $E_z^q[1] = E_z^{q-2}[2]$, nous allons l'appliquer au champ magnétique car elles sont supposées être le plus à gauche selon la grille de Yee (même chose pour les conditions de Dirichlet). Elles peuvent être implémentées en Matlab de la sorte :

```

1  %% Boucle principale
2  for T = 1 : Steps

```



```

3      % Mise à jour de B par E :
4      for nx = 1:Nx-1
5          By(nx) = By(nx) - mBy*(Ez(nx+1) - Ez(nx))/dx;
6      end
7      By(Nx) = By(Nx) - mBy*(E3 - Ez(Nx))/dx;
8      B3 = B2; B2 = B1; B1 = By(1);
9
10     % Mise à jour de E par B :
11     Ez(1) = mEz1(1)*Ez(1) - mEz2(1)*(By(1) - B3)/dx;
12     for nx = 2:Nx
13         Ez(nx) = mEz1(nx)*Ez(nx) - mEz2(nx)*(By(nx) - By(nx-1))/dx;
14     end
15     E3 = E2; E2 = E1; E1 = Ez(Nx);
16
17 end

```

Les conditions aux limites nulles, dites de Dirichlet, annulent mathématiquement le champ aux bords du domaine ce qui crée une onde régressive. Cela se traduit physiquement par un rebond de l'onde sur un mur parfait, qui renvoie l'onde en sens inverse. Un exemple de ce phénomène est illustré en Figure 1. Ces conditions peuvent être mises en œuvre à l'aide du code suivant :

```

1  %% Boucle principale
2  for T = 1 : Steps
3      % Mise à jour de B par E :
4      for nx = 1:Nx-1
5          By(nx) = By(nx) - mBy*(Ez(nx+1) - Ez(nx))/dx;
6      end
7      By(Nx) = By(Nx) - mBy*(E3 - Ez(Nx))/dx;
8      B3 = B2; B2 = B1; B1 = 0;
9
10     % Mise à jour de E par B :
11     Ez(1) = mEz1(1)*Ez(1) - mEz2(1)*(By(1) - B3)/dx;
12     for nx = 2:Nx
13         Ez(nx) = mEz1(nx)*Ez(nx) - mEz2(nx)*(By(nx) - By(nx-1))/dx;
14     end
15     E3 = E2; E2 = E1; E1 = 0;
16 end

```

Remarque : Appliquer les conditions aux limites uniquement à E_z revient plus ou moins au même en réalité mais ce n'est que par pure convention.

1.7 Sources

Pour ce qui est des sources, il suffit d'ajouter une source à un point ponctuel (E_{source} dans ce cas) dépendant uniquement du temps. Pour ce qui est de la source sinusoïdale, nous avons :

$$\mathbf{E}_{source}(t) = \sin(\omega t) \quad (12)$$

Pour ce qui est de la source Gaussienne, nous avons :

$$E_{source}(t) = \exp\left(-\left(\frac{t-t_0}{\tau}\right)^2\right) \quad (13)$$

Remarque : La source Gaussienne n'est pas centrée en $t = 0$ pour éviter un changement trop rapide de E et B au début de la simulation, et pour obtenir la gaussienne complète.

Implémentée en Matlab, elle donne :

```

1  % Source Gaussienne :
2  tau = 0.5/(fmax);      % Taille de l'impulsion (tau = 1/(pi*fmax) approximé)
3  t0 = 6*tau;           % Origine temporelle de la source décalée pour éviter une pente trop forte au début des
   ↳ itérations
4  g = @(t) exp(-((t-t0)/tau).^2);
5
6  % Source sinusoïdale :
7  w = 2*pi*fmax;        % Impulsion de la source
8  f = @(t) sin(w*t);

```

Et l'injection de ce terme dans l'équation de E_z se fait juste après la mise à jour des champs dans la boucle principale :

```

1  %% Boucle principale
2  for T = 1 : Steps
3      %calcul des champs
4      ...
5      % Source
6      Ez(3) = Ez(3) + f(T*dt);
7  end

```

1.8 Affichage Matlab

Pour ce qui est des affichages Matlab (plot, figure, etc.), nous nous sommes aidés de ChatGPT[8] pour améliorer le visuel des simulations et comprendre le fonctionnement des différentes fonctions. Cela sort du contexte du cours, donc nous n'allons pas nous attarder sur cette partie-là. Voici l'implémentation des affichages du champ électrique (ainsi que celles des obstacles) :

```

1  % Affichage du graphe :
2  h_plot=plot(x,Ez,'r','LineWidth',2);
3  hold on;
4  axis([0 max(x) -2 2]);
5  title('Champ électrique Ez (diélectrique avec pertes) ');
6  xlabel('x (m)');
7  ylabel('Ez (V/m)')
8  y_min = ylim;
9  vert = [0.4 0.8 0.4] ;      % Perte
10 rouge = [0.8 0.2 0.2];      % Diélectrique pur
11 bleu = [0.3 0.6 1];         % Quelconque
12 x_rect = [x_obs_d, x_obs_f, x_obs_f, x_obs_d]*dx;
13 y_rect = [y_min(1), y_min(1), y_min(2), y_min(2)];
14 h_rect = fill(x_rect, y_rect, bleu, 'FaceAlpha', 0.5, 'EdgeColor', 'none');

```

Ainsi qu'une partie dans la boucle, dont la fonction `set()` nous permet de ne mettre à jour que le champ Ez sans écraser ou modifier le reste de la figure ou du plot. La fonction `pause()` nous permet de choisir la vitesse à laquelle la simulation va se dérouler, et `drawnow` force la figure à dessiner Ez à chaque itération :

```

1  %% Boucle principale
2  for T = 1 : Steps
3      %Calcul des champs
4      ...
5      set(h_plot, 'YData', Ez); % ou re-plot(...) si besoin
6      pause(0.01)
7      drawnow;
8  end

```

Résultat et simulation 1D

Simulation avec une source gaussienne (conditions aux limites de Dirichlet) :

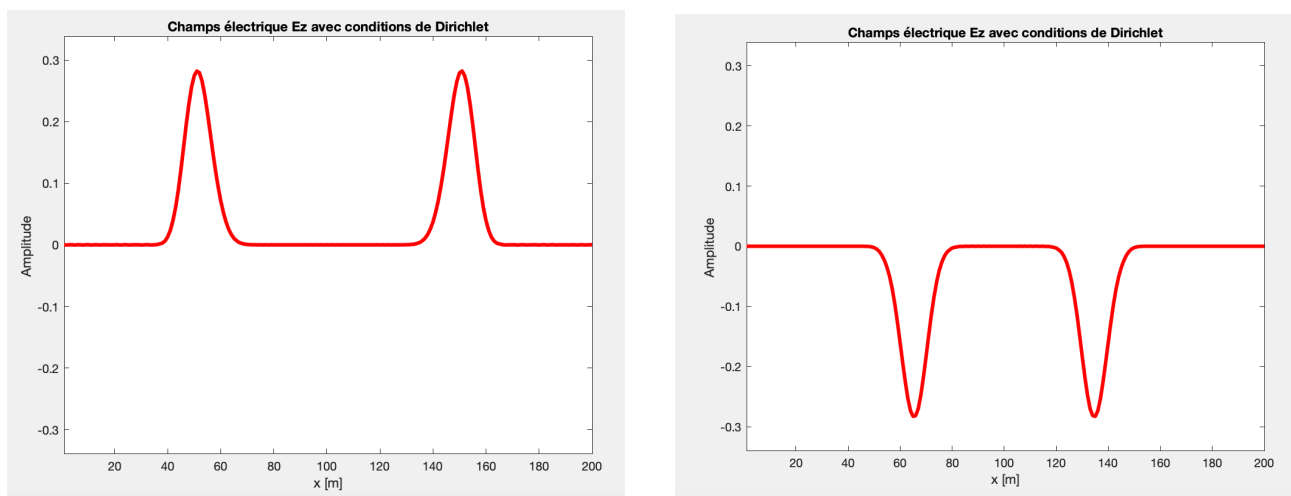


FIGURE 1 – Simulation d'une source gaussienne, conditions aux limites de Dirichlet

Simulation avec une source sinusoïdale (Conditions aux limites absorbantes) :

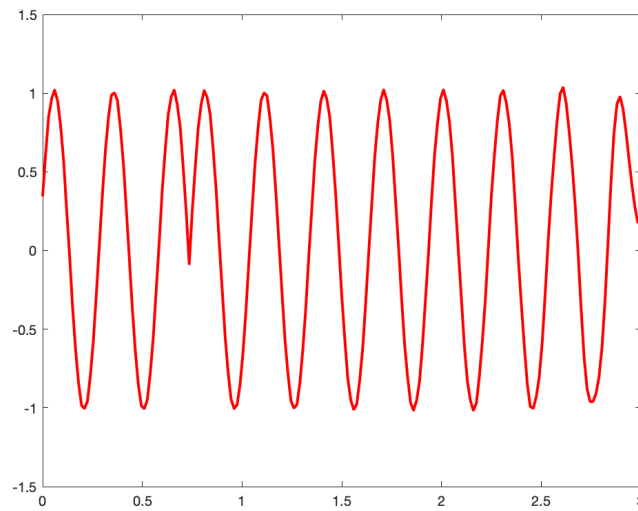


FIGURE 2 – Simulation d’une source sinusoïdale, conditions aux limites absorbantes

Dans notre configuration FDTD 1D, la source sinusoïdale est injectée avec une amplitude normalisée à 1 dans un milieu sans pertes (espace libre). En régime permanent et en l’absence de réflexions, le module du phaseur reste constant à 1 le long du domaine de simulation, conformément à la propagation d’une onde plane idéale.

Lorsqu’on ajoute une zone diélectrique pure, caractérisée par une permittivité relative $\epsilon_r > 1$ et une conductivité nulle ($\sigma = 0$), on observe une modification de la propagation de l’onde ainsi qu’une diminution de l’amplitude au contact de la zone diélectrique, car une partie du champ est réfléchi. Le diélectrique est représenté ici par la zone rouge. En introduisant un diélectrique avec pertes ($\epsilon_r > 1$ et $\sigma > 0$), représenté par la zone bleue, on observe une atténuation supplémentaire du champ électrique. Le champ décroît alors de manière exponentielle à l’intérieur de cette zone, comme illustré sur la figure suivante :

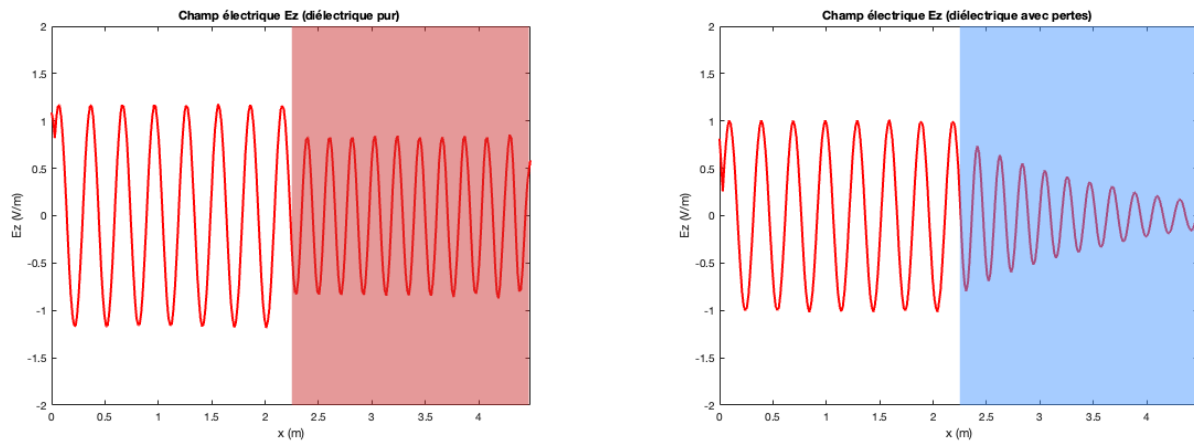


FIGURE 3 – Simulation d'une source à travers un diélectrique pur (à gauche) et un diélectrique avec pertes (à droite)

2 FDTD 2D

Dans cette section, nous allons expliciter nos méthodes d'implémentation de FDTD 2D dans le domaine de l'électromagnétisme. Pour ce faire, nous n'allons maintenant plus parler de vecteurs (comme c'était le cas pour la 1D) pour ce qui est de la grille, mais de matrices (de dimensions $N_x \times N_y$). Celles-ci représentent le plan spatial xy . Les champs utilisés (eux aussi sous forme de matrices) seront ceux de type TE (voir cours), c'est-à-dire : E_z , $\tilde{B}_x$ et $\tilde{B}_y$. **Remarque :** *Le champ $\tilde{B}$ est bien le champ magnétique B normalisé que l'on retrouve dans la section 1.*

2.1 Paramètres physiques

Les paramètres en 2D sont les mêmes que ceux de la 1D 1.1 :

```

1 %% Paramètres physiques
2
3 % Paramètres dans le vide :
4 c0 = 3e8;           % Vitesse de la lumière [m/s]
5 e0 = 8.8e-12;       % Permittivité du vide [F/m]
6 u0 = 4*pi*1e-7;     % Perméabilité du vide [H/m]
7
8 % Paramètres du matériau :
9 er = 4;             % Permittivité relative du matériau
10 ur = 1;             % Perméabilité relative du matériau
11 sigma_e = 5e-3;     % Conductivité électrique du matériau [S/m]
12 sigma_m = 0;        % Conductivité magnétique du matériau [S/m]
13
14 % Fréquence [Hz] :
15 fmax = 1e9;

```

2.2 Dimensionnement de la grille

Pour ce qui est de la stabilité de la grille spatiale, nous savons (voir consignes du projet ou [7]) que les pas spatiaux sont bornés de la sorte :

$$\Delta x, \Delta y \leq \frac{\lambda_{\min}}{N} \quad \begin{array}{l} \text{où } \lambda_{\min} \text{ est la plus petite longueur d'onde à représenter,} \\ \text{et } N \text{ est le nombre de points par longueur d'onde (typiquement } N \geq 10). \end{array} \quad (14)$$

Ensuite, nous entrons les dimensionnements de N_x et N_y voulus et, si ceux-ci ne correspondent pas aux critères de stabilité, ils sont modifiés de sorte à les satisfaire (lignes 21 à 28 du bloc de code suivant) :

```

1 %% Paramètres de la grille
2
3 % Longueur d'onde minimale [m] :
4 lambda_min = c0/fmax;

```

```

5
6 % Définition spatiale :
7 Nt = 10 ; % Car la bonne pratique impose Nt > 6 mais pas plus grand que 20 pour limiter la charge de
  ↳ calcul
8 delta_max = lambda_min/Nt; % Pour vérifier la stabilité du système
9
10 % Taille réelle de la grille [m] :
11 Lx = 10;
12 Ly = 10;
13
14 % Résolution des mailles dans la grille spatiale 1xGrid :
15 Nx = 200; % Nombre de mailles en x
16 Ny = 200; % Nombre de mailles en y
17 dx = Lx/Nx; % Pas spatial en x
18 dy = Ly/Ny; % Pas spatial en y
19
20 if dx > delta_max
21     dx = delta_max; % Pour éviter des instabilités
22     Nx = ceil(Lx/dx); % Adapter Nx
23 end
24 if dy > delta_max
25     dy = delta_max; % Pour éviter des instabilités
26     Ny = ceil(Ly/dy); % Adapter Ny
27 end

```

2.3 Paramètres de temps

De même pour le pas temporel (Source : [3]), il nous faudra le conditionner de sorte à satisfaire ce que l'on appelle les conditions de Courant, que voici :

$$\Delta t \leq \frac{1}{c \sqrt{\left(\frac{1}{\Delta x}\right)^2 + \left(\frac{1}{\Delta y}\right)^2}} \quad \text{où } c \text{ est la célérité maximale de l'onde dans les milieux,} \quad (15)$$

Δx et Δy sont les pas spatiaux en x et y .

```

1 % Définition temporelle :
2 dt = 1/(2*c0*sqrt((1/dx)^2 + (1/dy)^2)); % Le pas temporel en suivant les règles de bonne pratique
3 Steps = ceil(T_tot/dt); % Nombre d'itérations temporelles

```

Le T_{tot} (équation 7) est fixé de manière arbitraire, de façon à permettre l'observation des phénomènes d'intérêt. Un point de repère, issu de la sous-section 1.4 de l'étude 1D, a également été choisi à titre indicatif.

```

1 %% Paramètres de temps
2
3 % Temps de la simulation totale et nombre d'étapes :
4 N_max = max(Nx, Ny);
5 t_prop = (N_max*delta_max/10)/c0; % Temps de propagation d'un côté à l'autre de la grille
6 T_tot = 12*tau + 4*t_prop;

```

2.4 Équations de Maxwell en 2D

On considère un cas en 2D (plans x et y), avec un champ électrique $\mathbf{E}_z$ et des champs magnétiques $\tilde{\mathbf{B}}_x$ et $\tilde{\mathbf{B}}_y$ non nuls (rappel : $\tilde{\mathbf{B}} = c \cdot \mathbf{B}$, le champ magnétique normalisé). Les équations de

Maxwell sont donc (Sur base de [9]) :

$$\frac{\partial \tilde{\mathbf{B}}_y}{\partial x} - \frac{\partial \tilde{\mathbf{B}}_x}{\partial y} = c\epsilon\mu \frac{\partial \mathbf{E}_z}{\partial t} + c\mu J_z = \frac{\epsilon_r\mu_r}{c} \frac{\partial \mathbf{E}_z}{\partial t} + c\mu J_z \quad (16)$$

$$\frac{\partial \mathbf{E}_z}{\partial y} = -\frac{1}{c} \frac{\partial \tilde{\mathbf{B}}_x}{\partial t} \quad (17)$$

$$-\frac{\partial \mathbf{E}_z}{\partial x} = -\frac{1}{c} \frac{\partial \tilde{\mathbf{B}}_y}{\partial t} \quad (18)$$

En discrétisant avec des pas dans l'espace Δx et Δy et un pas dans le temps Δt avec la méthode de Yee, on retrouve les champs magnétiques :

$$\begin{aligned} \left. \frac{\partial \tilde{\mathbf{B}}_y}{\partial x} \right|_{m,n}^{q+1/2} &\approx \frac{\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2, n] - \tilde{\mathbf{B}}_y^{q+1/2}[m-1/2, n]}{\Delta x} \\ \left. \frac{\partial \tilde{\mathbf{B}}_x}{\partial y} \right|_{m,n}^{q+1/2} &\approx \frac{\tilde{\mathbf{B}}_x^{q+1/2}[m, n+1/2] - \tilde{\mathbf{B}}_x^{q+1/2}[m, n-1/2]}{\Delta y} \end{aligned}$$

Ainsi qu'en utilisant la même approximation qu'au chapitre de la 1D pour J_z (voir 1.2) :

$$J_z^{q+\frac{1}{2}}[m, n] = \sigma \mathbf{E}_z^{q+\frac{1}{2}}[m, n] \approx \sigma \cdot \frac{\mathbf{E}_z^{q+1}[m, n] + \mathbf{E}_z^q[m, n]}{2} \quad (19)$$

Après développement (voir chapitre théorique de la 1D 1.2), on retrouve le champ électrique :

$$\mathbf{E}_z^{q+1}[m, n] = \left(\frac{1 - \frac{\sigma \Delta t}{2\epsilon}}{1 + \frac{\sigma \Delta t}{2\epsilon}} \right) \mathbf{E}_z^q[m, n] +$$

$$\left(\frac{c\Delta t}{\epsilon_r\mu_r(1 + \frac{\sigma \Delta t}{2\epsilon})} \right) \left[\frac{\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2, n] - \tilde{\mathbf{B}}_y^{q+1/2}[m-1/2, n]}{\Delta x} - \frac{\tilde{\mathbf{B}}_x^{q+1/2}[m, n+1/2] - \tilde{\mathbf{B}}_x^{q+1/2}[m, n-1/2]}{\Delta y} \right] \quad (20)$$

Et finalement, les champs B_x et B_y sont mis à jour à $q+1/2$:

$$\tilde{\mathbf{B}}_x^{q+1/2}[m, n+1/2] = \tilde{\mathbf{B}}_x^{q-1/2}[m, n+1/2] - c\Delta t \cdot \frac{\mathbf{E}_z^q[m, n+1] - \mathbf{E}_z^q[m, n]}{\Delta y} \quad (21)$$

$$\tilde{\mathbf{B}}_y^{q+1/2}[m+1/2, n] = \tilde{\mathbf{B}}_y^{q-1/2}[m+1/2, n] + c\Delta t \cdot \frac{\mathbf{E}_z^q[m+1, n] - \mathbf{E}_z^q[m, n]}{\Delta x} \quad (22)$$

Vecteurs courbes d'évolution des champs

Dans le cadre d'une formulation compacte et exploitable en code, nous exprimons les mises à jour des champs comme des **vecteurs courbes**, c'est-à-dire des expressions discrètes représentant la contribution temporelle et directionnelle des champs électromagnétiques. Ces vecteurs seront donc calculés à chaque itération temporel.

Vecteur courbe $\mathbf{C}_y^E$: Ce vecteur décrit la variation de $\mathbf{E}_z$ sur Δx :

$$\mathbf{C}_y^E = \frac{\mathbf{E}_z^q[m+1, n] - \mathbf{E}_z^q[m, n]}{\Delta x}$$

En Matlab (Boucle principale) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      ...
5
6      % Construction du vecteur courbe CEx :
7      CEx(:,1:Ny-1) = (Ez(:,2:Ny) - Ez(:,1:Ny-1)) / dy;
8
9      ...
10
11 end

```

Vecteur courbe $\mathbf{C}_x^E$: Ce vecteur décrit la variation de $\mathbf{E}_z$ sur Δy :

$$\mathbf{C}_x^E = \frac{\mathbf{E}_z^q[m, n+1] - \mathbf{E}_z^q[m, n]}{\Delta y}$$

En Matlab (Boucle principale) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      ...
5
6      % Construction du vecteur courbe CEx :
7
8      ...
9
10     % Construction du vecteur courbe CEy :
11     CEy(1:Nx-1,:) = (Ez(2:Nx,:) - Ez(1:Nx-1,:)) / dx;
12
13     ...
14
15 end

```

Vecteur courbe $\mathbf{C}_z^B$: Ce vecteur décrit l'évolution sur Δx et Δy du champ magnétique $\tilde{\mathbf{B}}$:

$$\mathbf{C}_z^B = \frac{\tilde{\mathbf{B}}_x^{q+\frac{1}{2}}[m+\frac{1}{2}, n] - \tilde{\mathbf{B}}_x^{q+\frac{1}{2}}[m-\frac{1}{2}, n]}{\Delta x} - \frac{\tilde{\mathbf{B}}_y^{q+\frac{1}{2}}[m, n+\frac{1}{2}] - \tilde{\mathbf{B}}_y^{q+\frac{1}{2}}[m, n-\frac{1}{2}]}{\Delta y}$$

En Matlab (Boucle principale) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      ...
5
6      % Construction du vecteur courbe CEx :
7
8      ...
9
10     % Construction du vecteur courbe CEy :
11
12     ...
13
14     % Construction du vecteur courbe CBz (nx > 1, ny > 1) :
15     CBz(2:Nx,2:Ny) = (By(2:Nx,2:Ny) - By(1:Nx-1,2:Ny)) / dx ...
16                     - (Bx(2:Nx,2:Ny) - Bx(2:Nx,1:Ny-1)) / dy;
17
18     ...
19
20 end

```

Remarque : On remarque que dans les implémentations Matlab des vecteurs courbes, certains points (mêmes colonnes et lignes) ne sont pas inclus dans les calculs. Or, ceux-ci sont calculés et expliqués plus tard dans le chapitre des conditions aux bords 2.7 en 2D.

2.5 Mise à jour du champ E_z

L'équation discrète pour le champ électrique E_z , en tenant compte d'un matériau avec conductivité σ , est donnée par :

$$\mathbf{E}_z^{q+1}[n, m] = m_{E_z,1} \cdot \mathbf{E}_z^q[n, m] + m_{E_z,2} \cdot \mathbf{C}_z^B \quad (23)$$

Et son implémentation en Matlab (Boucle principale) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      ...
5
6      % Construction du vecteur courbe CEx :
7
8      ...
9
10     % Construction du vecteur courbe CEy :
11
12     ...
13
14     % Construction du vecteur courbe CBz (nx > 1, ny > 1) :
15
16     ...
17
18     % Mise à jour de Ez
19     Ez = mEz1.*Ez + mEz2.*CBz;
20
21     ...
22
23 end

```

Définitions des coefficients :

— Coefficient de pertes électriques (2D) :

$$m_{E_z,1} = \frac{1 - \frac{\sigma \Delta t}{2\varepsilon}}{1 + \frac{\sigma \Delta t}{2\varepsilon}}$$

— Coefficient lié à la variation spatiale de $\tilde{\mathbf{B}}$ dans la mise à jour de E_z (2D) :

$$m_{E_z,2} = \frac{c \Delta t}{\varepsilon_r \mu_r \left(1 + \frac{\sigma \Delta t}{2\varepsilon}\right)}$$

Ce qui donne l'implémentation suivante :

```

1  % Calcul des coefficients du champ Ez :
2  mEz1 = (1 - (sig_E*dt)./(2*ERz*e0))./(1 + (sig_E*dt)./(2*ERz*e0));
3  mEz2 = c0*dt./(URx.*ERz.*(1 + dt*sig_E./(2*e0*ERz)));

```

2.6 Mise à jour des champs $\tilde{\mathbf{B}}_x$ et $\tilde{\mathbf{B}}_y$

On établit l'équation discrète pour $\tilde{\mathbf{B}}_x$ et $\tilde{\mathbf{B}}_y$ en introduisant une conductivité magnétique σ_m , analogue au cas électrique. Sans signification physique ici, elle permet simplement d'appliquer uniformément les "PML" aux trois composantes (E_z , $\tilde{\mathbf{B}}_x$, $\tilde{\mathbf{B}}_y$) et est nulle en dehors de ces zones (voir 2.8). Les équations associées sont :

$$\begin{aligned} \tilde{\mathbf{B}}_x^{q+\frac{1}{2}}[n, m] &= m_{B_x,1} \cdot \tilde{\mathbf{B}}_x^{q-\frac{1}{2}}[n, m] - m_{B_x,2} \cdot \mathbf{C}_x^E \\ \tilde{\mathbf{B}}_y^{q+\frac{1}{2}}[n, m] &= m_{B_y,1} \cdot \tilde{\mathbf{B}}_y^{q-\frac{1}{2}}[n, m] + m_{B_y,2} \cdot \mathbf{C}_y^E \end{aligned} \quad (24)$$

Et leur implémentation en Matlab (Boucle principale) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      ...
5
6      % Construction du vecteur courbe CEx :
7
8      ...
9
10     % Construction du vecteur courbe CEy :
11
12     ...
13
14     % Mettre à jour B :
15     Bx = mBx1.*Bx - mBx2.*CEx;
16     By = mBy1.*By + mBy2.*CEy;
17
18     ...
19

```

```

20 % Construction du vecteur courbe CBz (nx > 1, ny > 1) :
21
22 ...
23
24 % Mise à jour de Ez
25
26 ...
27
28 end

```

Définitions des coefficients :

— Coefficients de pertes magnétiques (PML) :

$$m_{B_x,1} = m_{B_y,1} = \frac{1 - \frac{\sigma_m \Delta t}{2\mu}}{1 + \frac{\sigma_m \Delta t}{2\mu}}$$

— Coefficients liés à la variation spatiale de E_z (PML) :

$$m_{B_x,2} = m_{B_y,2} = \frac{c \Delta t}{\left(1 + \frac{\sigma_m \Delta t}{2\mu}\right)}$$

Ce qui donne l'implémentation suivante :

```

1 % Calcul des coefficients du champ Bx :
2 mBx1 = (1 - (sig_Mx*dt)./(2*u0*URx))./(1 + (sig_Mx*dt)./(2*u0*URx));
3 mBx2 = c0*dt./(1 + dt*sig_Mx./(2*u0*URx));
4
5 % Calcul des coefficients du champ By :
6 mBy1 = (1 - (sig_My*dt)./(2*u0*URy))./(1 + (sig_My*dt)./(2*u0*URy));
7 mBy2 = c0*dt./(1 + dt*sig_My./(2*u0*URy));

```

Remarque : Lorsque σ_m est nul (ce qui est le cas partout sauf dans les zones "PML"), les coefficients sont égaux à ceux trouvés en 1D (c'est-à-dire : $m_{B_x,1} = m_{B_y,1} = 1$ et $m_{B_x,2} = m_{B_y,2} = c \cdot \Delta t$).

2.7 Conditions aux bords

En FDTD 2D, on utilisera les conditions de Dirichlet (c'est-à-dire : valeurs nulles des champs aux bords du domaine). Celles-ci ne sont plus applicables au début et à la fin d'un vecteur comme dans le cas 1D, mais sur les premières et dernières lignes et colonnes des champs. Pour les mêmes raisons qu'en 1D (voir 1.3), on applique les conditions aux bords des premières lignes et colonnes au champ magnétique $\tilde{\mathbf{B}}$, et les conditions aux bords des dernières lignes et colonnes au champ électrique $\mathbf{E}$:

Pour ce qui est des conditions aux courbes sur $\tilde{\mathbf{B}}_x$ et $\tilde{\mathbf{B}}_y$, on passe par les vecteurs courbes $\mathbf{C}_x^E$ et $\mathbf{C}_y^E$ (Voir 2.4, Source : [3]) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      % Construction du vecteur courbe CEx :
5
6      ...
7
8      % Dernière colonne (ny = Ny, nx > 1)
9      CEx(:,Ny) = (0-Ez(:,Ny)) / dy;
10
11     ...
12
13     % Construction du vecteur courbe CEy :
14
15     ...
16
17     % Dernière ligne (nx = Nx, ny > 1)
18     CEy(Nx,:) = (Ez(Nx,:)- 0)/ dx;
19
20     ...
21
22     % Mettre à jour B :
23
24     ...
25
26     % Construction du vecteur courbe CBz (nx > 1, ny > 1) :
27
28     ...
29
30     % Mise à jour de Ez
31
32     ...
33
34 end
35
36
37

```

Et pour ce qui est des conditions aux limites sur E_z , on passe par le vecteur courbe C_z^B (Voir 2.4, Source : [3]) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      ...
5
6      % Construction du vecteur courbe CEx :
7
8      ...
9
10     % Construction du vecteur courbe CEy :
11
12     ...
13
14     % Mettre à jour B :
15
16     ...
17
18     % Coin supérieur gauche
19     CBz(1,1) = (By(1,1) - 0)/dx - (Bx(1,1) - 0)/dy;
20
21     % Première colonne (ny = 1, nx > 1)
22     CBz(2:Nx,1) = (By(2:Nx,1) - By(1:Nx-1,1)) / dx - (Bx(2:Nx,1)) / dy;
23
24     % Première ligne (nx = 1, ny > 1)
25     CBz(1,2:Ny) = (By(1,2:Ny) - 0) / dx - (Bx(1,2:Ny) - Bx(1,1:Ny-1)) / dy;
26
27     % Rest du vecteur courbe CBz (nx > 1, ny > 1) :
28
29     ...

```

```

30
31     % Mise à jour de Ez
32
33     ...
34
35 end

```

2.8 Perfect Match Layer (PML)

Des conditions aux bords absorbantes comme dans le cas de la 1D (voir 1.3) ne sont pas possibles, mais il est possible d'utiliser une combine mathématique afin d'éviter que les ondes de nos champs atteignent les bords et y rebondissent pour ensuite venir interférer avec les phénomènes que l'on cherche à observer. Cette artifice s'appelle le PML et consiste à atténuer progressivement (pas d'un coup, pour éviter les réflexions) les champs lorsqu'ils se rapprochent des bords (c'est-à-dire lorsqu'ils rentrent dans les zones PML que nous allons définir plus loin).

Même si ceci n'était pas demandé dans les consignes du projet, nous avons tout de même fait le choix d'inclure un PML dans notre code pour deux raisons principales : pouvoir placer la source où l'on veut, sans contrainte, et ne pas avoir à faire de grilles trop grandes. La 2ème raison est de loin la plus contraignante car en 2D, simuler une grande grille revient à un très long temps de calcul, ce qui donne des simulations très longues à chaque fois, et c'est tout simplement très lourd pour le processeur de l'ordinateur. De plus, c'est tout simplement plus agréable pour les yeux d'utiliser un PML.

Pour ce qui est du point de vue mathématique, une vidéo plus détaillée sur les PML est proposée dans la bibliographie [4]. Le PML utilisé dans notre code n'est pas le plus optimal et reste assez simple (l'entièreté de l'onde n'est pas entièrement absorbée). Nous avons choisi un PML d'une épaisseur égale à 5% de la grille. Ces couches se situent aux 4 bords de la grille et ont une épaisseur de $0.05 \cdot Nx$ sur le bord haut et le bord bas, et une épaisseur de $0.05 \cdot Ny$ sur le bord gauche et le bord droit.

L'atténuation dans les coefficients des sous-sections 2.6 et 2.7 se fera via les conductivités électrique et magnétique (σ et σ_m) qui varieront selon la relation :

$$\sigma_x(x) = \begin{cases} 0, & \text{si } x \in \text{domaine physique} \\ \sigma_{\max} \left(\frac{x}{d} \right)^m, & \text{si } x \in [0, d] \\ \sigma_{\max} \left(\frac{L-x}{d} \right)^m, & \text{si } x \in [L-d, L] \end{cases} \quad (25)$$

L'équation 25 nous donne les bords gauche et droit. On applique le même principe pour le

bord haut et le bord bas :

$$\sigma_y(y) = \begin{cases} 0, & \text{si } y \in \text{domaine physique} \\ \sigma_{\max} \left(\frac{y}{d} \right)^m, & \text{si } y \in [0, d] \\ \sigma_{\max} \left(\frac{H-y}{d} \right)^m, & \text{si } y \in [H-d, H] \end{cases} \quad (26)$$

Sous forme indicielle, et en prenant arbitrairement une relation d'ordre $m = 3$:

$$\sigma_x(nx) = \begin{cases} 0, & \text{si } nx \in \text{domaine physique} \\ \sigma_{\max} \left(\frac{nx}{0.05 \cdot Nx} \right)^3, & \text{si } nx \in [1, 0.05 \cdot Nx] \\ \sigma_{\max} \left(\frac{Nx-nx}{0.05 \cdot Nx} \right)^3, & \text{si } nx \in [Nx-0.05 \cdot Nx, Nx] \end{cases} \quad (27)$$

et

$$\sigma_y(ny) = \begin{cases} 0, & \text{si } ny \in \text{domaine physique} \\ \sigma_{\max} \left(\frac{ny}{0.05 \cdot Ny} \right)^3, & \text{si } ny \in [1, 0.05 \cdot Ny] \\ \sigma_{\max} \left(\frac{Ny-ny}{0.05 \cdot Ny} \right)^3, & \text{si } ny \in [Ny-0.05 \cdot Ny, Ny] \end{cases} \quad (28)$$

Il reste à déterminer $\sigma_{\max}$. Or, celui-ci doit atténuer de la manière la plus homogène possible les coefficients des sous-sections 2.6 et 2.7. Pour s'en assurer, on trouve la relation :

$$\sigma_{\max} = \begin{cases} \frac{k \cdot \epsilon_0}{\Delta t}, & \text{pour } \sigma \\ \frac{k \cdot \mu_0}{\Delta t}, & \text{pour } \sigma_m \end{cases} \quad (29)$$

k est à déterminer expérimentalement. Dans notre cas, $k = 0.7$ semble bien fonctionner, et l'implémentation du PML donne donc :

```

1  %% Calcul des "Perfect Match Layer" (PML)
2
3  % Épaisseur de la PML:
4  NPML = floor([Nx, Nx, Ny, Ny]*0.05);    % [Bord_gauche, Bord_droit, Bord_sup, Bord_inf]
5
6  % Calcul du paramètre absorbant en x :
7  sig_E = SIGMA_E;
8  sig_Mx = SIGMA_Mx;
9  sig_My = SIGMA_My;
10 for nx = 1 : NPML(1)
11     nx1 = NPML(1) - nx + 1;
12     sig_E(nx1,:) = (0.7*epsilon0/dt)*(nx/NPML(1))^3;
13     sig_Mx(nx1,:) = (0.7*u0/dt)*(nx/NPML(1))^3;
14     sig_My(nx1,:) = (0.7*u0/dt)*(nx/NPML(1))^3;
15 end
16 for nx = 1 : NPML(2)
17     nx1 = Nx - NPML(2) + nx;
18     sig_E(nx1,:) = (0.7*epsilon0/dt)*(nx/NPML(2))^3;
19     sig_Mx(nx1,:) = (0.7*u0/dt)*(nx/NPML(2))^3;

```

```

20     sig_My(nx1,:) = (0.7*u0/dt)*(nx/NPML(2))^3;
21 end
22
23 % Calcul du paramètre absorbant en y :
24 for ny = 1 : NPML(3)
25     ny1 = NPML(3) - ny + 1;
26     sig_E(:,ny1) = (0.7*e0/dt)*(ny/NPML(3))^3;
27     sig_Mx(:,ny1) = (0.7*u0/dt)*(ny/NPML(3))^3;
28     sig_My(:,ny1) = (0.7*u0/dt)*(ny/NPML(3))^3;
29 end
30 for ny = 1 : NPML(4)
31     ny1 = Ny - NPML(4) + ny;
32     sig_E(:,ny1) = (0.7*e0/dt)*(ny/NPML(4))^3;
33     sig_Mx(:,ny1) = (0.7*u0/dt)*(ny/NPML(4))^3;
34     sig_My(:,ny1) = (0.7*u0/dt)*(ny/NPML(4))^3;
35 end

```

Où sig_E , sig_Mx , et sig_My sont respectivement les matrices de conductivité électrique, conductivité magnétique appliquée à $\tilde{\mathbf{B}}_x$ et conductivité magnétique appliquée à $\tilde{\mathbf{B}}_y$. Pour rappel, dans notre code, la conductivité magnétique n'a pas de signification physique et vaut zéro tout le temps en dehors des zones PML.

2.9 Sources

Pour ce qui est des sources, rien ne change par rapport à la simulation en 1D, si ce n'est que l'emplacement de la source est maintenant en 2D :

```

1  %% Calculs des sources
2
3  % Localisation source :
4  nx_source = ceil(Nx/8);
5  ny_source = ceil(Ny/2);
6
7  % Source Gaussienne :
8  tau = 0.5/(1e9);    % Taille de l'impulsion (tau = 1/(pi*fmax) approximé)
9  t0 = 3*tau;         % Origine temporelle de la source décalée pour éviter une pente trop forte au début des
    ↳ itérations
10 g = @(t) exp(-((t-t0)/tau).^2);
11
12 % Source sinusoïdale :
13 w = 2*pi*1e9;       % Impulsion de la source
14 f = @(t) sin(w*t);

```

On l'applique dans la boucle principale après la mise à jour de $\mathbf{E}$ et $\tilde{\mathbf{B}}$ de la sorte :

```

1  %% Boucle principale
2  for T = 1 : Steps
3
4      %Calcul des champs
5      ...
6      % Injection de la source :
7      Ez(nx_source,ny_source) = Ez(nx_source,ny_source) + f(T*dt);
8

```

2.10 Obstacles

Les obstacles choisis sont soit des rectangles, soit des formes rondes (ellipse ou cercle), soit des demi-plans. Pour ce qui est des manipulations de matrices, nous nous sommes inspirés des vidéos [5] et [6] afin de dessiner ces différentes formes géométriques. On ne va pas rentrer dans les détails de ces manipulations car cela sort du cadre du projet. Ce qu'il faut retenir, c'est que l'on attribue aux matrices ER_z , UR_x , UR_y et $SIGMA_E$ (respectivement les matrices de permittivité relative, perméabilité relative appliquée à $\tilde{\mathbf{B}}_x$, perméabilité relative appliquée à $\tilde{\mathbf{B}}_y$ et conductivité électrique) leurs valeurs dans la grille. Ces matrices sont donc de dimensions $N_x \times N_y$. Partout en dehors de l'obstacle, elles ont les propriétés du vide (sauf la conductivité dans les zones PML). OBS1 et OBS3 serviront à afficher visuellement les contours des obstacles. On a donc le bout de code suivant :

```

1  %% Dimensionnement des obstacles
2
3  OBS1 = zeros(Nx,Ny);
4  OBS3 = zeros(Nx,Ny);
5
6  % Dimensions des obstacles rectangulaires (m) :
7  wx = 8;
8  wy = 1;
9
10 % Paramètres spatiaux des obstacles rectangulaires :
11 nx = round(wx/dx);           % Taille en indice matriciel (en x) de l'obstacle
12 nx1 = 1 + floor(Nx/2);       % Positionnement des points gauches de l'obstacle en indice matriciel
13 nx2 = nx1 + round(nx/2) - 1; % Positionnement des points droits de l'obstacle en indice matriciel
14
15 ny = round(wy/dy);           % Taille en indice matriciel (en y) de l'obstacle
16 ny1 = 1 + round((Ny - ny)/2 - 2.5/dx); % Positionnement des points supérieurs de l'obstacle en indice matriciel
17 ny2 = ny1 + ny - 1;          % Positionnement des points inférieurs de l'obstacle en indice matriciel
18
19 % La grille centrée :
20 xa = [0 : Nx - 1]*dx;
21 xa = xa - mean(xa);
22
23 ya = [0 : Ny - 1]*dy;
24 ya = ya - mean(ya);
25
26 % Paramètres spatiaux des obstacles ronds :
27 xc = 2.5;
28 yc = 0;
29 rx = 1;
30 ry = 1;
31 [Y, X] = meshgrid(ya, xa);
32
33 % Type d'obstacle utilisé :
34 rect = 1;
35 rond = 0;
36 half = 0;
37
38 % Mise à jour de la grille pour les obstacles :
39 if rect == 1
40     ERz(nx1:nx2, ny1:ny2) = 1;
41     URx(nx1:nx2, ny1:ny2) = ur;
42     URy(nx1:nx2, ny1:ny2) = ur;
43     SIGMA_E(nx1:nx2, ny1:ny2) = 0;
44     OBS1(nx1:nx2, ny1:ny2) = 1;
45 end
46 if half == 1
47     for ny = 1 : Nx
48         nx_limit = floor(Nx/2);
49         ERz(nx_limit:Nx, ny) = er;

```

```
50         URx(nx_limit:Nx, ny) = ur;
51         URy(nx_limit:Nx, ny) = ur;
52         SIGMA_E(nx_limit:Nx, ny) = sigma_e;
53         OBS3(nx_limit:Nx, ny) = 1;
54     end
55 end
56 if rond == 1
57     G = (((X - xc) / rx).^2 + ((Y - yc) / ry).^2) <= 1;
58     ERz(G) = er2;
59     URx(G) = ur;
60     URy(G) = ur;
61     SIGMA_E(G) = sigma_e;
62     OBS1(G) = 1;
63 end
```

2.11 Puissance et tension captées par une antenne de hauteur équivalente et résistance connues

Pour cette sous-section, nous considérons une antenne dipôle de demi-longueur d'onde ($\lambda/2$) placée sur l'axe $\vec{I}_z$. Les hypothèses suivantes sont posées :

- L'approximation du champ lointain est valable donc seul la composante transverse (ici $\vec{I}_z$) sera significative pour le champ E, et sa décroissance sera proportionnel à $1/\sqrt{r}$ (équation 4.35 du cours [2]).
- L'onde incidente est parfaitement polarisée selon l'axe $\vec{I}_z$, correspondant à la polarisation de l'antenne réceptrice.
- l'onde se propage dans le plan xy , l'angle θ vaut donc 90 degrés.
- L'impédance de l'antenne est adaptée à la puissance absorbée par la charge, on considère donc une puissance collecté maximal $Z_l = Z_a^*$.

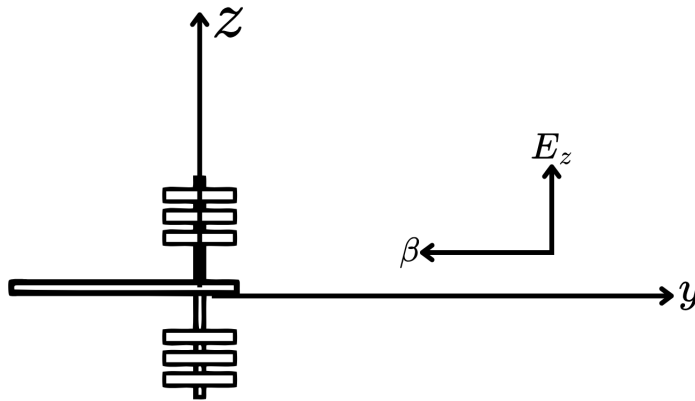


FIGURE 4 – Antenne en réception avec une incidence maximal

En conséquence, le champ électrique incident en l'origine s'écrit :

$$\vec{E}_i(0) = E_i e^{j\vec{\beta} \cdot \vec{0}} \vec{I}_z = E_z \vec{I}_z. \quad (30)$$

En toute généralité, l'expression de la hauteur équivalente d'une antenne dipôle est donnée par (voir cours [2]) :

$$\vec{h}_e(\theta, \phi) = \frac{\lambda * \cos(\pi/2 \cos \theta)}{\pi \sin^2 \theta} \vec{I}_z \quad (31)$$

La tension captée par une antenne dipôle $\lambda/2$ est donnée par la relation (Voir cours [2]) :

$$\underline{V}_{oc}(0) = -\vec{h}_e(\theta, \phi) \cdot \vec{E}_i = -\frac{\lambda}{\pi} E_i \frac{\cos\left(\frac{\pi}{2} \cos\theta\right)}{\sin\theta} \quad (32)$$

Or on a considéré que $\theta = 90^\circ$ donc :

$$V_{oc} = -\frac{\lambda}{\pi} E_z \quad (33)$$

Dans notre simulation, on travaille à 1GHz donc la longueur d'onde est $\lambda = c/f = 0,3m$. La tension réceptionnée sera donc donnée par :

$$V_{oc} \approx -0,095 \cdot E_z \quad (34)$$

La puissance captée par une antenne dipôle $\lambda/2$ est donnée par la relation (Voir cours [2]) :

$$P_L = \frac{|\underline{V}_{oc}|^2}{8R_a} = \frac{h_e^2 |\underline{E}_z|^2}{8R_a} = 1.5e-5 \cdot |\underline{E}_z|^2 \quad (35)$$

On a pris $R_a = 73\Omega$ qui est une valeur standard.

Sur Matlab (fin de la boucle principale) :

```

1  %% Boucle principale
2  for T = 1 : Steps
3      %Calcul des champs
4      ...
5      % Mise à jour de Voc
6      Voc = -0,095*Ez;
7      % Mise à jour de PL
8      PL = 1.5e-5*Ez.*Ez;
9  
```

2.12 Affichage et enregistrement Matlab

Tout comme en 1D (Voir sous-section 1.8), nous avons utilisé Chatgpt[8] pour embellir nos simulations et dessiner les contours de nos obstacles (avec la fonction *contour()*). De plus, en 2D, nous avons ajouté la possibilité d'enregistrer, sous fichiers, nos simulations (avec la fonction *VideoWriter()*). Voici le bloc de code utilisé :

```

1  %% Affichages graphiques
2
3  fig = figure;
4  set(fig, 'Name', 'FDTD 2D');
5  set(fig, 'NumberTitle', 'off');
6  h1 = pcolor(X0', Y0', Ez);
7  shading interp
8  axis equal tight
9  colorbar
10 caxis([1e-6, 0.1])

```

```

11 xlabel('x (m)');
12 ylabel('y (m)');
13 title('Ez');
14 xticks(linspace(0, 9, 10));
15 yticks(linspace(0, 9, 10));
16 hold on;
17 contour(X0', Y0', OBS1, [0.5 0.5], 'k', 'LineWidth', 2);
18 contour(X0', Y0', OBS3, [0.5 0.5], 'Color',[0.3 0.3 0.3], 'LineWidth', 2);
19
20 %% Enregistrement de la simulation
21
22 % Animation :
23 v = VideoWriter('fdtd_simulation.avi'); % Nom du fichier de sortie
24 v.FrameRate = 20; % Taux d'images par seconde
25 open(v);

```

Tout ça étant placé avant la boucle principale. Pour ce qui se trouve dedans :

```

1 %% Boucle principale
2 for T = 1 : Steps
3
4     ...
5
6     % Représentation graphique :
7     set(h1, 'CData', Ez); % Mettre à jour uniquement les données de CData de pcolor
8     drawnow
9     frame = getframe(fig);
10    writeVideo(v, frame);
11 end

```

Et pour ce qui se passe après la boucle principale (on arrête tout simplement l'enregistrement) :

```

1 %% Fin de l'enregistrement de la simulation
2
3 close(v);

```

Résultat et simulation 2D

Propagation d'une onde sinusoïdale dans le vide

La Figure 5 présente une simulation FDTD en deux dimensions d'une onde électromagnétique sinusoïdale émise par une source ponctuelle. Le champ considéré est polarisé selon l'axe z (champ E_z), et la source est localisée au centre du domaine simulé. La propagation se fait dans le vide, sans obstacles ni matériaux.

Même dans un milieu sans pertes (vide), on observe une décroissance apparente de l'amplitude de l'onde au fur et à mesure de sa propagation. Cette décroissance n'est pas due à une dissipation d'énergie, mais au phénomène de dispersion géométrique : l'énergie se répartit sur une surface de plus en plus grande à mesure que l'onde se propage radialement. En deux dimensions, cela se traduit par une diminution de l'amplitude en $1/\sqrt{r}$, où r est la distance à la source. Cette décroissance est expliquée par la formule suivante du champ rayonné par un fil infiniment long (voir [2]) :

$$\vec{E} = -j\omega\vec{A} \simeq -Z_0\sqrt{\frac{j\beta}{8\pi}}\frac{e^{-j\beta r}}{\sqrt{r}}I_{1z} \quad (36)$$

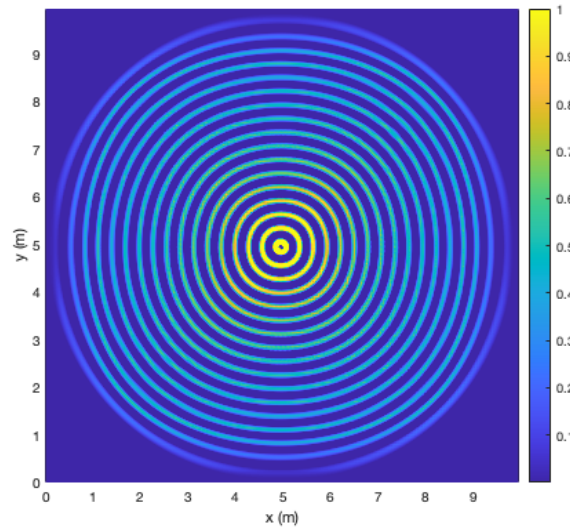


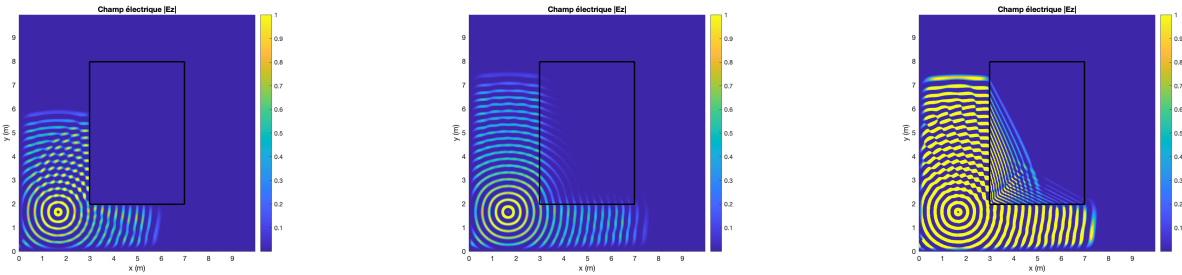
FIGURE 5 – Simulation FDTD 2D : onde sinusoïdale dans le vide

Propagation d'une onde sinusoïdale à travers un diélectrique

En pratique, l'utilisation de valeurs très élevées pour la conductivité σ dans les simulations FDTD conduit à des instabilités numériques. En effet, les termes de conduction σ et de permittivité ϵ dans les équations de mise à jour du champ électrique et magnétique (équations 23 et 24) peuvent dominer l'évolution du champ et provoquer une décroissance trop rapide ou une instabilité. Pour contourner ce problème dans nos simulations, une valeur modérée de σ a été choisie tout en assurant un comportement représentatif. On observe alors qu'en présence d'un milieu fortement conducteur, l'onde électromagnétique est entièrement réfléchie à l'interface et ne pénètre pas dans le carré, comme illustré en Figure 13a. Ce phénomène est typique d'un conducteur parfait.

En diminuant la conductivité tout en conservant une permittivité relative élevée, l'onde pénètre partiellement dans le matériau, mais subit une forte atténuation. Ce phénomène est visible sur la Figure 13b.

Enfin, lorsqu'on simule un matériau diélectrique avec des pertes et une permittivité $\epsilon_r > 1$, l'onde est atténuée et sa longueur d'onde est réduite à l'intérieur du matériau, comme le montre la Figure 13c. Ce comportement est cohérent avec la relation $\lambda = \lambda_0 / \sqrt{\epsilon_r}$.



(a) Simulation 2D d'un rectangle bon conducteur

(b) Simulation 2D d'un rectangle faiblement conducteur

(c) Simulation 2D d'un rectangle diélectrique avec pertes

FIGURE 6 – Comparaison de différente propagation à travers des matériaux de conductivité différentes.

Puissance et tension réceptionnées par une antenne dipôle $\lambda/2$ dans le vide

Ici, nous avons la simulation des tensions et puissances réceptionnées, calculées en tous points de l'espace pour une résistance R_a et une hauteur équivalente $\vec{h}_e$ connues (Voir sous-section 2.11 pour les calculs).

On remarque que l'amplitude de la tension V_{oc} est simplement proportionnelle au champ élec-

trique E_z (équation 34), tandis que la puissance P_L perd en amplitude beaucoup plus rapidement, en raison de sa proportionnalité au carré du champ électrique E_z (équation 35). La figure 7 montre la valeur instantanée des champs, de la tension et de la puissance captée. La figure 8 illustre la plus grande valeur de l'amplitude des phaseurs sur toute la durée de la simulation.

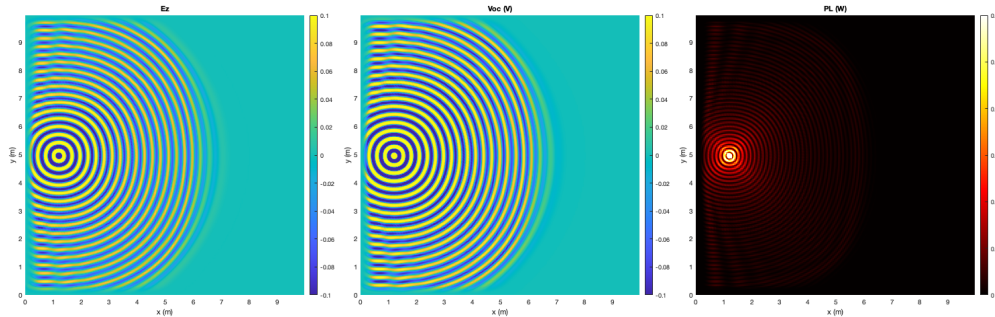


FIGURE 7 – Comparaison champ électrique E_z , tension V_{oc} et puissance P_L réceptionnées par une antenne avec une résistance R_a et une hauteur équivalente $\vec{h}_e$ connues, simulation dans le vide

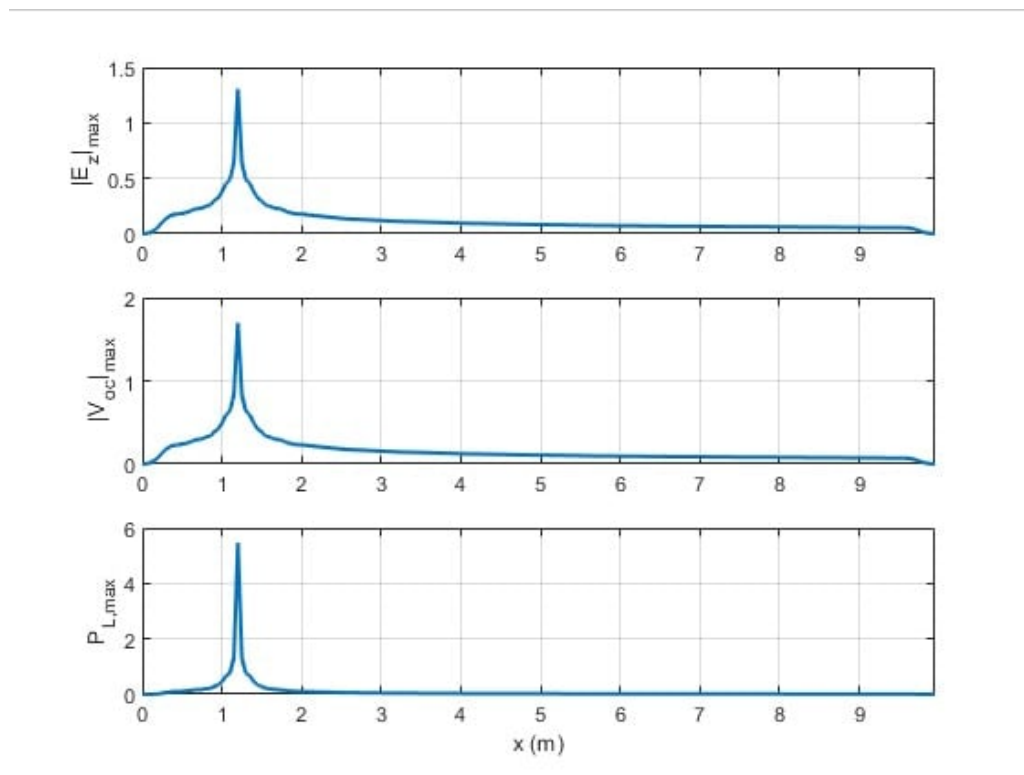


FIGURE 8 – Amplitude maximal des phaseurs de la tension et de la puissance captée, on remarque que V_{oc} décroît linéairement et P_L quadratiquement par rapport à E_z (voir équations 34, 35).

On Remarque la décroissance en $1/\sqrt{r}$ de l'amplitude maximal du phaseur de E_z prédit par l'équation (36). Ce qui est différent du cas 1D (toujours dans le vide) où le phaseur était constant et valait 1 le long du domaine de simulation (2).

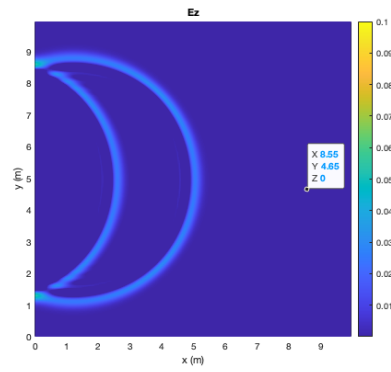
Conditions de Dirichlet en 2D (sans PML)

FIGURE 9 – Conditions de Dirichlet 2D sans PML

Illustration du cours

Simulations d'ondes électromagnétiques à travers un milieux diélectriques et conducteurs

Cette section, basée sur le chapitre 7, porte sur la **simulation des ondes électromagnétiques dans différents milieux** et sur l'**interprétation physique des phénomènes observés**.

Pour rappel (voir [2]), l'onde du champ électrique $\mathbf{E}$ est donnée par :

$$\underline{\mathbf{E}}(\vec{r}) = \underline{\mathbf{E}}_0 e^{-\vec{\gamma} \cdot \vec{r}} = \underline{\mathbf{E}}_0 e^{-\vec{\alpha} \cdot \vec{r}} e^{-j\vec{\beta} \cdot \vec{r}}$$

où $\vec{\gamma} = \vec{\alpha} + j\vec{\beta}$ est la **constante de propagation complexe**.

Dans le domaine temporel :

$$\mathbf{E}(\vec{r}, t) = \mathbf{E}_0 e^{-\vec{\alpha} \cdot \vec{r}} \cos(\omega t - \vec{\beta} \cdot \vec{r})$$

avec $\omega t - \vec{\beta} \cdot \vec{r} = \text{constante}$.

La **décroissance exponentielle** de l'amplitude est gouvernée par le paramètre α , qui s'écrit en général(voir [2]) :

$$\alpha = \omega \sqrt{\frac{\mu_0 \epsilon}{2}} \left[\sqrt{1 + \left(\frac{\sigma}{\omega \epsilon} \right)^2} - 1 \right]^{1/2} \quad (37)$$

Milieu bon conducteur

Dans un **bon conducteur**, où $\sigma \gg \omega\epsilon$:

$$\alpha \approx \beta \approx \sqrt{\frac{\omega\mu\sigma}{2}} \quad (38)$$

Lorsque $\sigma > 0$, la partie réelle α de γ entraîne une **décroissance exponentielle** de l'amplitude le long de la direction de propagation. On obtient ainsi une **onde plane inhomogène** dont l'amplitude décroît selon :

$$E(z) = E_0 e^{-\alpha z} \quad \text{avec } \alpha \approx \sqrt{\frac{\omega\mu\sigma}{2}}$$

Cette expression illustre la **dissipation du champ dans un bon conducteur**.

C'est exactement ce que l'on observe dans la simulation (figure 10), où le champ électrique E_z est représenté en bleu et l'exponentielle décroissante en rouge pointillé. Pour la simulation, nous avons utilisé l'expression générale de α (37), car l'approximation (38) ne permet pas un calcul suffisamment précis. En effet, la simulation FDTD sur MATLAB présente certaines limites à ce niveau. Nous avons donc dû modéliser un conducteur avec une conductivité faible ($\sigma = 2.85 \cdot 10^{-3} \text{ S/m}$) afin de rester dans les conditions où la formule de base de α reste valide.

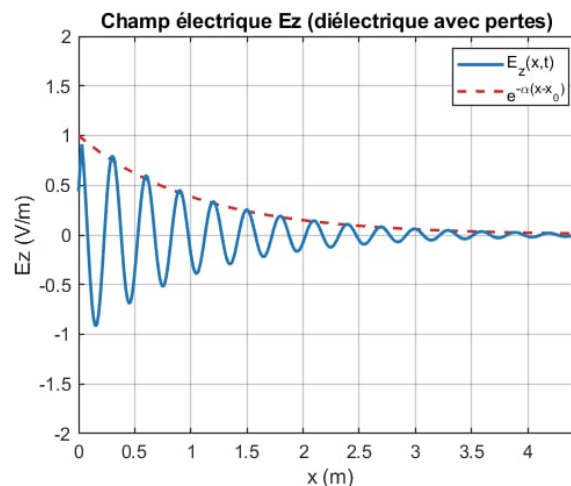


FIGURE 10 – Décroissance de l'amplitude du champ électrique dans un milieu avec pertes ($\sigma = 2.85 \cdot 10^{-3} \text{ S/m}$, $f = 1 \text{ GHz}$).

La fonction exponentielle sur matlab est implémentée comme tel :

```
1 % Exponentielle négative :
2 alpha = 0.9545; % alpha = 2*pi*f*mu*sqrt(u0*sigma_e*u0/2)*(sqrt(1+(sigma_e/(2*pi*f*mu*epsilon_e))^2) - 1)^(1/2)
3 e = @(x) exp(-alpha*(x));
```

Milieu diélectrique sans perte

Les paramètres d'un courant électrique $\vec{E}$ d'un milieu diélectrique sans perte sont (voir cours) :

$$\alpha = 0$$

$$\beta = \omega \sqrt{\mu_0 \epsilon}$$

Les milieux diélectriques ayant une permittivité ϵ supérieure à celle du vide, passer du vide à un tel milieu aura pour effet d'augmenter β et donc de diminuer la vitesse de phase v_φ , étant donné que :

$$v_\varphi = \frac{\omega}{\beta}$$

Ce phénomène est illustré par la figure 11 :

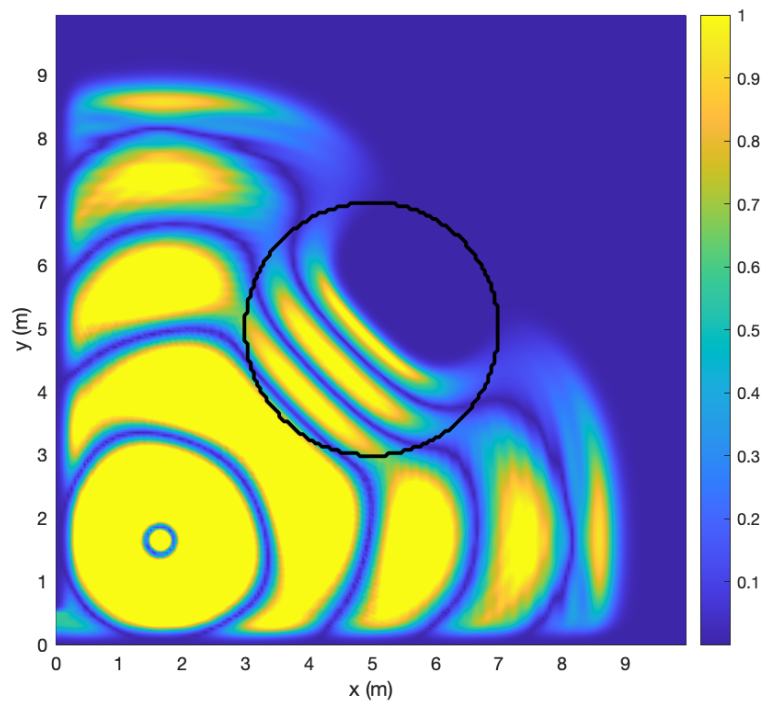


FIGURE 11 – Ralentissement de la vitesse de phase, dû au diélectrique sans perte (cercle)

2.13 Réflexion/Transmission à travers un mur

Pour ce deuxième chapitre (8), nous avons décidé d'illustrer la diffraction de l'onde sur un mur de conductivité et de permittivité différentes de celles du vide, l'onde est polarisé perpendiculairement au mur d'épaisseur l comme illustré dans le figure 12. Le mur, caractérisé par sa permittivité ϵ_m sans pertes, génère des réflexions multiples internes décrites par la série géométrique suivante :

$$E_r = \Gamma_{\perp}(\theta_i)E_i + \sum_{n=1}^{\infty} (1 + \Gamma_{\perp}(\theta_i))(-\Gamma_{\perp}(\theta_i))^{2n-1}(1 - \Gamma_{\perp}(\theta_i))e^{-j2\beta_m ns}e^{j\beta 2ns \sin^2 \theta_i} E_i \quad (39)$$

où $\Gamma_{\perp}$ est donné par l'équation suivante (voir[2]) :

$$\Gamma_{\perp} = \frac{Z_2 \cos \theta_t - Z_1 \cos \theta_i}{Z_2 \cos \theta_t + Z_1 \cos \theta_i}$$

Les coefficients de transmission à travers le mur (T_m) et de réflexion (Γ_m) sont (voir[2]) :

$$\Gamma_m(\theta_i) = \Gamma_{\perp}(\theta_i) - (1 - \Gamma_{\perp}^2(\theta_i)) \frac{\Gamma_{\perp}(\theta_i) e^{-2j\beta_m s} e^{j\beta 2s \sin \theta_i \sin \theta_i}}{1 - \Gamma_{\perp}^2(\theta_i) e^{-2j\beta_m s} e^{j\beta 2s \sin \theta_i \sin \theta_i}} \quad (40)$$

$$T_m(\theta_i) = \frac{(1 - \Gamma_{\perp}^2(\theta_i)) e^{-j\beta_m s}}{1 - \Gamma_{\perp}^2(\theta_i) e^{-2j\beta_m s} e^{j\beta 2s \sin \theta_i \sin \theta_i}} \quad (41)$$

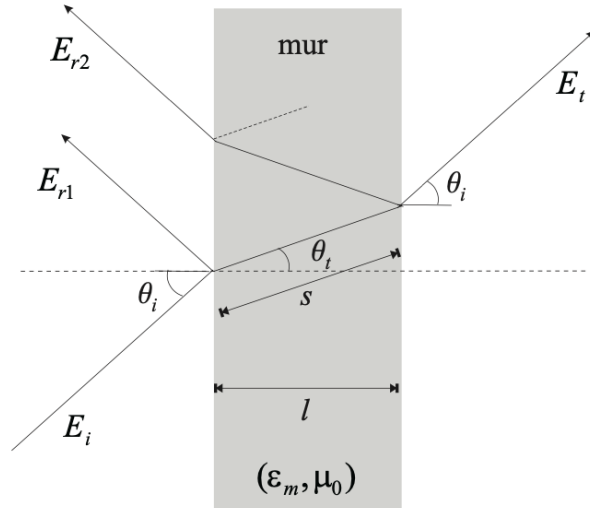


Figure 8.4 – Transmission et réflexion par un mur d'épaisseur l .

Pour notre simulation nous avons considerer la fréquence $f = 1$ GHz ($\lambda_0 = 0.3$ m dans le vide, $\varepsilon_1 = 1$) incidente sur un mur d'épaisseur $l = 1.5$ m ($\approx 5\lambda_0$) et de permittivité $\varepsilon_m = 4$. Pour $\theta_i = 0^\circ$.

On remarque sur la figure 13 ci-dessous que le comportement est tout à fait prédit : l'onde bleue est bien la réflexion de nos ondes incidentes sur le mur. On peut même voir sur la 3^e image que, lorsque l'onde passe le mur, celle-ci est réfléchiée et transmise à nouveau (futurs E_{r2} et E_t sur la figure 12).

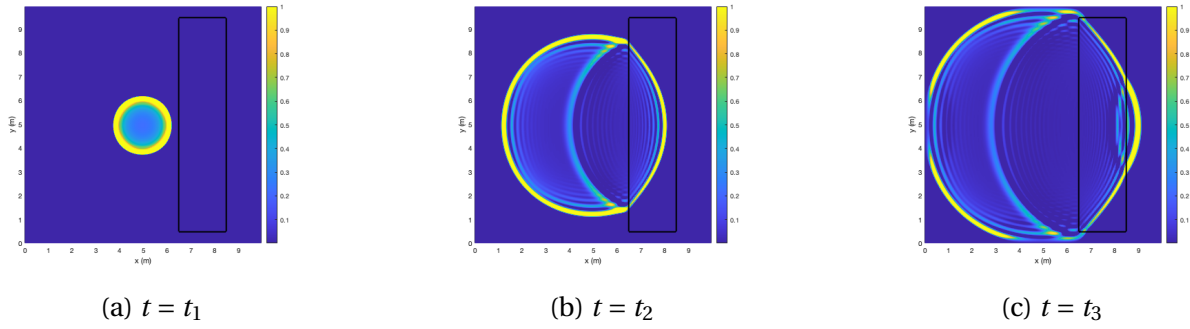


FIGURE 13 – Évolution de la diffraction à travers un mur de longueur l

2.14 Propagation guidée

Pour la simulation du chapitre 9, nous avons choisi d'étudier la propagation d'une onde plane à l'intérieur d'un guide d'ondes modélisé comme deux plaques conductrices parallèles. Cette configuration permet d'observer clairement les réflexions multiples des ondes électromagnétiques entre les parois du guide.

Le champ résultant entre les deux plaques s'écrit (voir [2]) :

$$\vec{E} = 2jE_0 \sin(\beta y \cos \theta) e^{-j\beta z \sin \theta} \vec{\mathbf{i}}_x$$

avec les conditions aux limites de Dirichlet imposant :

$$\beta b \cos \theta = m\pi$$

On obtient ainsi une condition sur l'angle d'incidence en fonction du mode :

$$\theta_n = \arccos\left(\frac{m\pi}{\beta b}\right), \quad \beta = \frac{2\pi f}{c}$$

Pour qu'un mode se propage, il faut que la fréquence de la source soit supérieure ou égale à sa fréquence de coupure :

$$f_{cn} = \frac{n \cdot c}{2b} \quad (42)$$

où :

- f_{cn} : fréquence de coupure du mode TE_n (Hz),
- n : numéro du mode,
- c : vitesse de la lumière dans le vide ($\approx 3 \times 10^8$ m/s),
- b : distance entre les deux plaques conductrices (m).

Pour nos simulations, nous avons illustré le premier mode pour une fréquence $f = 1$ GHz et un angle d'incidence θ_1 donné par :

$$\theta_1 = \arccos\left(\frac{\pi}{\beta b}\right) = 45^\circ$$

d'où :

$$b = \frac{c}{2f \cos 45^\circ} \approx 0.212 \text{ m}$$

La vitesse de groupe est alors :

$$v_g = c \sin \theta_1 \approx 2.12 \times 10^8 \text{ m/s} < c$$

et la vitesse de phase :

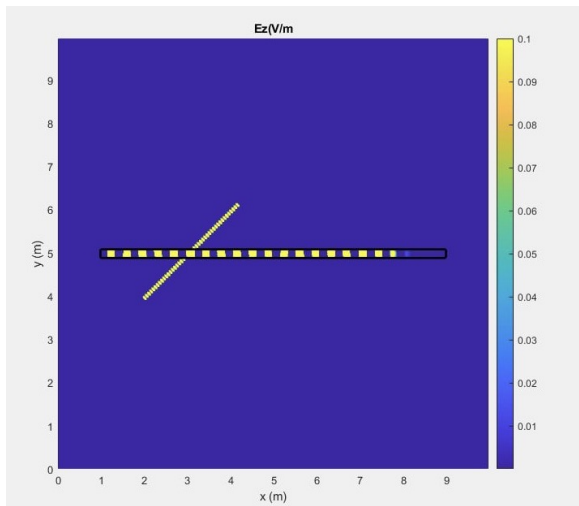
$$v_\phi = \frac{\omega}{\beta \sin \theta_1} \approx 4.24 \times 10^8 \text{ m/s} > c$$

ce qui n'est pas problématique, cette grandeur étant purement géométrique.

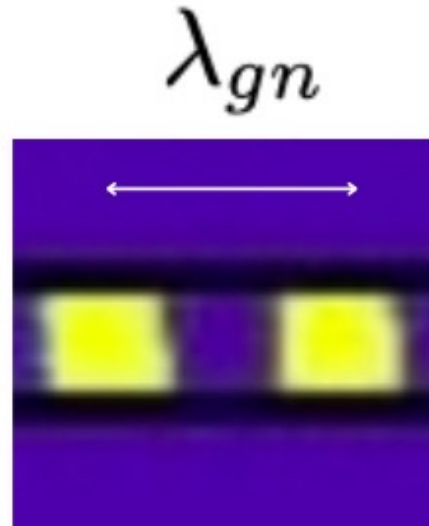
Enfin, la longueur d'onde géométrique vaut :

$$\lambda_{gn}^{\text{th}} = \frac{2\pi}{\beta \sin \theta_1} \approx 0.0424 \text{ m}$$

Une mesure directe sur la simulation (figure 14a) donne une valeur proche de 0.04 m.



(a) Propagation du premier mode, $b = 0.212 \text{ m}$, $f = 1 \text{ GHz}$, angle $\theta_1 = 45^\circ$.



(b) Calcul de la longueur d'onde géométrique

Code complet Matlab

1D

```

1  % Projet physique de télécommunication
2  % Etape 1 : FDTD 1D en espace libre
3  close all;
4  clc;
5  clear all;
6  %% Paramètres
7  % Paramètres du vide :
8  c0 = 3e8;           % Vitesse de la lumière [m/s]
9  e0 = 8.8e-12;       % Permittivité du vide [F/m]
10 u0 = 4*pi*1e-7;     % Perméabilité du vide [H/m]
11
12 % Paramètres du matériau :
13 er = 2;             % Permittivité relative du matériau
14 ur = 1;             % Perméabilité relative du matériau
15 nmax = 1;           % Index de refraction maximal (= 1 dans l'espace libre)
16 sigma_e = 5e-3;     % Conductivité électrique du matériau [S/m]
17 sigma_m = 0;        % Conductivité magnétique du matériau [S/m]
18
19 % Nombre de mailles dans les grilles spatiales :
20
21 Nx = 300;
22
23 % Initialisation des matériaux à l'espace libre :
24 ER = ones(1,Nx);
25 UR = ones(1,Nx);
26 SIGMA_E = zeros(1,Nx);
27
28 % fréquence maximale :
29 fmax = 1e9;
30
31 % Longueur d'onde minimale :
32 lambda_min = c0/(fmax*nmax);
33
34 % définitions spatiale et temporelle :
35 N = 20 ;           % Car la bonne pratique impose Nz_max > 6 mais pas plus grand que 20 pour limiter la charge
36   ↳ de calcul
37 dx = lambda_min/N; % Pour vérifier la stabilité du système
38 dt = nmax*dx/(2*c0); % En découle le pas temporel par bonne pratique
39
40 % Obstacle diélectrique ou conducteur :
41 x_obs_d = round(Nx/2); % Emplacement début obstacle
42 x_obs_f = Nx - 2;      % Emplacement fin obstacle
43 ER(x_obs_d:x_obs_f) = er;
44 UR(x_obs_d:x_obs_f) = ur;
45 SIGMA_E(x_obs_d:x_obs_f) = sigma_e;
46
47 % Source Gaussienne :
48 tau = 0.5/(fmax); % Taille de l'impulsion (tau = 1/(pi*fmax) approximé)
49 t0 = 6*tau;      % Origine temporelle de la source décalée pour éviter une pente trop forte au début des
50   ↳ itérations
51 g = @(t) exp(-(((t-t0)/tau).^2));
52
53 % Source sinusoidale :
54 w = 2*pi*fmax; % Impulsion de la source
55 f = @(t) sin(w*t);
56
57 % Temps de la simulation totale :
58 t_prop = (nmax*Nx*dx)/c0; % Temps de propagation d'un coté à l'autre de la grille
59 T_tot = 12*tau + 5*t_prop; % Formule de bonne pratique
60 Steps = ceil(T_tot/dt); % Nombre d'itération temporelle
61
62 % Calcul des coefficients (avec perte) :
63 mEz1 = (1 - SIGMA_E*dt./(2*e0*ER))./(1 + SIGMA_E*dt./(2*e0*ER));
64 mEz2 = c0*dt./(ER.*UR.*(1 + SIGMA_E*dt./(2*e0*ER)));
65 mBy = c0*dt;
66
67 % Initialisation de Hx et Ey :
68 By = zeros(1,Nx); % Utilisation de H pour représenter B (B = μ*H) pour aérer les calculs

```

```

68 Ez = zeros(1,Nx);
69
70 % Initialisation des conditions limites :
71 B3 = 0; B2=B3; B1=B2;
72 E3 = 0; E2=E3; E1=E2;
73
74 % Objet figure :
75 fig = figure('Color','w');
76 set(fig,'Name','FDTD 1D');
77 set(fig,'NumberTitle','off');
78
79 % Vecteur espace :
80 x = [0:Nx-1]*dx;
81
82 % Exponentielle négative :
83 alpha = 3.3545; % alpha = sqrt(2*pi*fmax*simu_e*u0/2)
84 e = @(x) exp(-alpha*(x-0.75));
85
86 % Vecteur temps :
87 t = [0:Steps-1]*dt;
88
89
90 % Affichage du graphe :
91 h_plot=plot(x,Ez,'r','LineWidth',2);
92 hold on;
93 axis([0 max(x) -2 2]);
94 title('Champ électrique Ez (diélectrique avec pertes) ');
95 xlabel('x (m)');
96 ylabel('Ez (V/m)')
97 y_min = ylim;
98 vert = [0.4 0.8 0.4] ; % Perte
99 rouge = [0.8 0.2 0.2]; % Dielectrique pur
100 bleu = [0.3 0.6 1]; % Quelconque
101 x_rect = [x_obs_d, x_obs_f, x_obs_f, x_obs_d]*dx;
102 y_rect = [y_min(1), y_min(1), y_min(2), y_min(2)];
103 h_rect = fill(x_rect, y_rect, bleu, 'FaceAlpha', 0.5, 'EdgeColor', 'none');
104
105
106
107 %% Boucle principale
108 for T = 1 : Steps
109     % Mise à jour de B par E :
110     for nx = 1:Nx-1
111         By(nx) = By(nx) - mBy*(Ez(nx+1) - Ez(nx))/dx;
112     end
113     By(Nx) = By(Nx) - mBy*(E3 - Ez(Nx))/dx;
114     B3 = B2; B2 = B1; B1 = By(1);
115
116     % Mise à jour de E par B :
117     Ez(1) = mEz1(1)*Ez(1) - mEz2(1)*(By(1) - B3)/dx;
118     for nx = 2:Nx
119         Ez(nx) = mEz1(nx)*Ez(nx) - mEz2(nx)*(By(nx) - By(nx-1))/dx;
120     end
121     E3 = E2; E2 = E1; E1 = Ez(Nx);
122     % Source
123     Ez(3) = Ez(3) + f(T*dt);
124     set(h_plot, 'YData', Ez); % ou re-plot(...) si besoin
125     pause(0.01)
126     drawnow;
127
128 end

```

2D

```

1 % Projet physique de télécommunication
2 % Etape 1 : FDTD 2D en espace libre
3 close all;
4 clc;
5 clear all;
6
7 %% Paramètres physiques
8

```

```

9  % Paramètres dans le vide :
10 c0 = 3e8;           % Vitesse de la lumière [m/s]
11 e0 = 8.8e-12;       % Permittivité du vide [F/m]
12 u0 = 4*pi*1e-7;     % Perméabilité du vide [H/m]
13
14 % Paramètres du matériau :
15 er = 4;             % Permittivité relative du matériau
16 er2 = 2;            % 2ème permittivité relative (au cas où)
17 ur = 1;             % Perméabilité relative du matériau
18 sigma_e = 5e-3;     % Conductivité électrique du matériau [S/m]
19 sigma_m = 0;        % Conductivité magnétique du matériau [S/m]
20
21 % Fréquence [Hz] :
22 fmax = 1e9;
23
24 %% Paramètres de la grille
25
26 % Longueur d'onde minimale [m] :
27 lambda_min = c0/fmax;
28
29 % Définition spatiale :
30 Nt = 2 ;            % Car la bonne pratique impose Nt > 6 mais pas plus grand que 20 pour limiter la charge de
    ↪ calcul
31 delta_max = lambda_min/Nt; % Pour vérifié la stabilité du système
32
33
34 % Taille réelle de la grille [m] :
35 Lx = 10;
36 Ly = 10;
37
38 % Résolution des mailles dans la grille spatiale iGrid :
39 Nx = 200;           % Nombre de maille en x
40 Ny = 200;           % Nombre de maille en y
41 dx = Lx/Nx;         % Pas spatial en x
42 dy = Ly/Ny;         % Pas spatial en y
43
44 if dx > delta_max
45     dx = delta_max; % Pour éviter des instabilités
46     Nx = ceil(Lx/dx); % Adapter Nx
47 end
48 if dy > delta_max
49     dy = delta_max; % Pour éviter des instabilités
50     Ny = ceil(Ly/dy); % Adapter Ny
51 end
52
53 % Initialisation des matrices des matériaux :
54 ERz = ones(Nx,Ny);
55 URx = ones(Nx,Ny);
56 URy = ones(Nx,Ny);
57 SIGMA_E = zeros(Nx,Ny);
58 SIGMA_Mx = zeros(Nx,Ny);
59 SIGMA_My = zeros(Nx,Ny);
60
61 %% Dimensionnement des obstacles
62
63 OBS1 = zeros(Nx,Ny);
64
65 OBS3 = zeros(Nx,Ny);
66
67 % Dimension des obstacles rectangulaires (m) :
68 wx = 8;
69 wy = 1;
70
71 % Paramètres spatiaux des obstacles rectangulaires :
72 nx = round (wx/dx) ; % Taille en indice matriciel (en x) de l'obstacle
73 nx1 = 1 + floor (Nx/2) ; % Posionnement des points gauches de l'obstacle en indice matriciel
74 nx2 = nx1 + round(nx/2) - 1; % Posionnement des points droits de l'obstacle en indice matriciel
75
76 ny = round (wy/dy) ; % Taille en indice matriciel (en y) de l'obstacle
77 ny1 = 1 + round ((Ny - ny) /2 - 2.5/dx); % Posionnement des points supérieurs de l'obstacle en indice
    ↪ matriciel
78 ny2 = ny1 + ny - 1; % Posionnement des points inférieurs de l'obstacle en indice matriciel
79
80 % La grille centrée :
81 xa = [0 : Nx - 1]*dx;
82 xa = xa - mean(xa);

```

```

83
84 ya = [0 : Ny - 1]*dy;
85 ya = ya - mean(ya);
86
87 % Paramètres spatiaux des obstacles ronds :
88 xc = 2.5;
89 yc = 0;
90 rx = 1;
91 ry = 1;
92 [Y, X] = meshgrid (ya, xa) ;
93
94 % Type d'obstacle utilisé :
95 rect = 0;
96 rond = 0;
97 half = 0;
98
99 % Mise à jour de la grille pour les obstacles :
100 if rect == 1
101     ERz(nx1:nx2, ny1:ny2) = 1;
102     URx(nx1:nx2, ny1:ny2) = ur;
103     URy(nx1:nx2, ny1:ny2) = ur;
104     SIGMA_E(nx1:nx2, ny1:ny2) = 0;
105     OBS1(nx1:nx2, ny1:ny2) = 1;
106 end
107 if half == 1
108     for ny = 1 : Nx
109         nx_limit = floor(Nx/2);
110         ERz(nx_limit:Nx, ny) = er;
111         URx(nx_limit:Nx, ny) = ur;
112         URy(nx_limit:Nx, ny) = ur;
113         SIGMA_E(nx_limit:Nx, ny) = sigma_e;
114         OBS3(nx_limit:Nx, ny) = 1;
115     end
116 end
117 if rond == 1
118     G = (((X - xc) /rx).^2 + ((Y - yc) /ry).^2) <= 1;
119     ERz(G) = er2;
120     URx(G) = ur;
121     URy(G) = ur;
122     SIGMA_E(G) = sigma_e;
123     OBS1(G) = 1;
124 end
125
126
127 %% Calculs des sources
128
129 % Localisation source :
130 nx_source = ceil(Nx/8);
131 ny_source = ceil(Ny/2);
132
133 % Source Gaussienne :
134 tau = 0.5/(1e9); % Taille de l'impulsion (tau = 1/(pi*fmax) approximé)
135 t0 = 3*tau; % Origine temporelle de la source décalée pour éviter une pente trop forte au début des
136 % itérations
137 g = @(t) exp(-(((t-t0)/tau).^2));
138
139 % Source sinusoidale :
140 w = 2*pi*1e9; % Impulsion de la source
141 f = @(t) sin(w*t);
142
143 %% Paramètres de temps
144
145 % Temps de la simulation totale et nombres d'étapes :
146 N_max = max(Nx,Ny);
147 t_prop = (N_max*delta_max/10)/c0; % Temps de propagation d'un coté a l'autre de la grille
148 T_tot = 12*tau + 4*t_prop; % Formule de bonne pratique
149
150 % Définition temporelle :
151 dt = 1/(2*c0*sqrt((1/dx)^2 + (1/dy)^2)); % Le pas temporel en suivant les règles de bonne pratique
152 Steps = ceil(T_tot/dt); % Nombre d'itération temporelle
153
154 %% Calcul des "Perfect Match Layer" (PML)
155
156 % Épaisseur de la PML:
157 NPML = floor([Nx, Nx, Ny, Ny]*0.05); % [Bord_gauche, Bord_droit, Bord_sup, Bord_inf]

```

```

158
159 % Calcul du paramètre absorbant en x :
160 sig_E = SIGMA_E;
161 sig_Mx = SIGMA_Mx;
162 sig_My = SIGMA_My;
163 for nx = 1 : NPML(1)
164     nx1 = NPML(1) - nx + 1;
165     sig_E(nx1,:) = (0.7*e0/dt)*(nx/NPML(1))^3;
166     sig_Mx(nx1,:) = (0.7*u0/dt)*(nx/NPML(1))^3;
167     sig_My(nx1,:) = (0.7*u0/dt)*(nx/NPML(1))^3;
168 end
169 for nx = 1 : NPML(2)
170     nx1 = Nx - NPML(2) + nx;
171     sig_E(nx1,:) = (0.7*e0/dt)*(nx/NPML(2))^3;
172     sig_Mx(nx1,:) = (0.7*u0/dt)*(nx/NPML(2))^3;
173     sig_My(nx1,:) = (0.7*u0/dt)*(nx/NPML(2))^3;
174 end
175
176 % Calcul du paramètre absorbant en y :
177 for ny = 1 : NPML(3)
178     ny1 = NPML(3) - ny + 1;
179     sig_E(:,ny1) = (0.7*e0/dt)*(ny/NPML(3))^3;
180     sig_Mx(:,ny1) = (0.7*u0/dt)*(ny/NPML(3))^3;
181     sig_My(:,ny1) = (0.7*u0/dt)*(ny/NPML(3))^3;
182 end
183 for ny = 1 : NPML(4)
184     ny1 = Ny - NPML(4) + ny;
185     sig_E(:,ny1) = (0.7*e0/dt)*(ny/NPML(4))^3;
186     sig_Mx(:,ny1) = (0.7*u0/dt)*(ny/NPML(4))^3;
187     sig_My(:,ny1) = (0.7*u0/dt)*(ny/NPML(4))^3;
188 end
189
190 %% Calcul de coefficients
191
192 % Calcul des coefficient du champ Bx :
193 mBx1 = (1 - (sig_Mx*dt)./(2*u0*URx))./(1 + (sig_Mx*dt)./(2*u0*URx));
194 mBx2 = c0*dt./(1 + dt*sig_Mx./(2*u0*URx));
195
196 % Calcul des coefficient du champ By :
197 mBy1 = (1 - (sig_My*dt)./(2*u0*URy))./(1 + (sig_My*dt)./(2*u0*URy));
198 mBy2 = c0*dt./(1 + dt*sig_My./(2*u0*URy));
199
200 % Calcul des coefficient du champ Ez :
201 mEz1 = (1 - (sig_E*dt)./(2*ERz*e0))./(1 + (sig_E*dt)./(2*ERz*e0));
202 mEz2 = c0*dt./(URx.*ERz.*(1 + dt*sig_E./(2*e0*ERz)));
203
204
205
206 %% Initialisation des vecteurs principaux :
207
208 % Initialisation des champs électro-magnétiques :
209 Ez = zeros(Nx,Ny);
210 Bx = zeros(Nx,Ny);
211 By = zeros(Nx,Ny);
212 CEx = zeros(Nx,Ny);
213 CEy = zeros(Nx,Ny);
214 CBz = zeros(Nx,Ny);
215 Voc = zeros(Nx,Ny);
216 PL = zeros(Nx,Ny);
217
218 %% Vecteurs x et y
219 x = [0:Nx-1]*dx;
220
221 y = [0:Ny-1]*dy;
222
223 [X0,Y0] = meshgrid(x,y);
224
225 %% Affichages graphique
226
227 fig = figure;
228 set(fig,'Name','FDTD 2D');
229 set(fig,'NumberTitle','off');
230
231 % --- Premier subplot : Ez ---
232 subplot(1,3,1); % 1 ligne, 3 colonnes, 1er graphe
233 h1 = pcolor(X0', Y0', Ez);

```

```

234 shading interp
235 axis equal tight
236 colorbar
237 caxis([1e-6, 0.1])
238 xlabel('x (m)');
239 ylabel('y (m)');
240 title('Ez');
241 xticks(linspace(0, 9, 10));
242 yticks(linspace(0, 9, 10));
243 hold on;
244 %contour(X0', Y0', OBS1, [0.5 0.5], 'k', 'LineWidth', 2);
245 %contour(X0', Y0', OBS3, [0.5 0.5], 'Color',[0.3 0.3 0.3], 'LineWidth', 2);
246
247 % --- Deuxième subplot : Tension captée ---
248 ax2 = subplot(1,3,2); % 2e graphe
249 h2 = pcolor(X0', Y0', Voc); % À adapter selon ce que tu veux afficher
250 shading interp
251 axis equal tight
252 colorbar
253 colormap(ax2, 'jet')
254 caxis([1e-6, 0.1])
255 xlabel('x (m)');
256 ylabel('y (m)');
257 title('Voc (V)');
258 xticks(linspace(0, 9, 10));
259 yticks(linspace(0, 9, 10));
260 hold on;
261 %contour(X0', Y0', OBS1, [0.5 0.5], 'k', 'LineWidth', 2);
262 %contour(X0', Y0', OBS3, [0.5 0.5], 'Color',[0.3 0.3 0.3], 'LineWidth', 2);
263
264 % --- Troisième subplot : Puissance captée ---
265 ax3 = subplot(1,3,3); % 3e graphe
266 h3 = pcolor(X0', Y0', PL); % À adapter également
267 shading interp
268 axis equal tight
269 colorbar
270 colormap(ax3, 'hot')
271 caxis([1e-6, 0.1])
272 xlabel('x (m)');
273 ylabel('y (m)');
274 title('PL (W)');
275 xticks(linspace(0, 9, 10));
276 yticks(linspace(0, 9, 10));
277 hold on;
278 %contour(X0', Y0', OBS1, [0.5 0.5], 'k', 'LineWidth', 2);
279 %contour(X0', Y0', OBS3, [0.5 0.5], 'Color',[0.3 0.3 0.3], 'LineWidth', 2);
280
281 %% Enregistrement de la simulation
282
283 % Animation :
284 v = VideoWriter('fdtd_simulation.avi'); % Nom du fichier de sortie
285 v.FrameRate = 20; % Taux d'images par seconde
286 open(v);
287
288 %% Boucle principale
289 for T = 1 : Steps
290     % Construction du vecteur courbe CEx :
291     CEx(:,1:Ny-1) = (Ez(:,2:Ny) - Ez(:,1:Ny-1)) / dy;
292     % Dernière colonne (ny = Ny, nx > 1)
293     CEx(:,Ny) = (0-Ez(:,Ny)) / dy;
294     % Construction du vecteur courbe CEy :
295     CEy(1:Nx-1,:) = (Ez(2:Nx,:) - Ez(1:Nx-1,:)) / dx;
296     % Dernière ligne (nx = Nx, ny > 1)
297     CEy(Nx,:) = (Ez(Nx,:)- 0)/ dx;
298     % Mettre à jour B :
299     Bx = mBx1.*Bx - mBx2.*CEx;
300     By = mBy1.*By + mBy2.*CEy;
301     % Construction du vecteur courbe CBz :
302     % Coin supérieur gauche
303     CBz(1,1) = (By(1,1) - 0)/dx - (Bx(1,1) - 0)/dy;
304     % Première colonne (ny = 1, nx > 1)
305     CBz(2:Nx,1) = (By(2:Nx,1) - By(1:Nx-1,1)) / dx - (Bx(2:Nx,1)) / dy;
306     % Première ligne (nx = 1, ny > 1)
307     CBz(1,2:Ny) = (By(1,2:Ny) - 0) / dx - (Bx(1,2:Ny) - Bx(1,1:Ny-1)) / dy;
308     % Reste de la matrice (nx > 1, ny > 1)
309     CBz(2:Nx,2:Ny) = (By(2:Nx,2:Ny) - By(1:Nx-1,2:Ny)) / dx ...

```

```

310         - (Bx(2:Nx,2:Ny) - Bx(2:Nx,1:Ny-1)) / dy;
311     % Mise à jour de Ez
312     Ez = mEz1.*Ez + mEz2.*CBz;
313     % Injection de la source :
314     Ez(nx_source,ny_source) = Ez(nx_source,ny_source) + f(T*dt);
315     % Mise à jour de Voc
316     Voc = -1.3*Ez;
317     % Mise à jour de PL
318     PL = 3.2*Ez.*Ez;
319     % Représentation graphique :
320     set(h1, 'CData', Ez); % Mettre à jour uniquement les données de CData de pcolor
321     set(h2, 'CData', Voc);
322     set(h3, 'CData', PL);
323     drawnow
324     frame = getframe(fig);
325     writeVideo(v, frame);
326 end
327
328 %% Fin de l'enregistrement de la simulation
329
330 close(v);

```

Bibliographie

- [1] Napov ARTEM. *MATH-H202 - Analyse numérique*. 2023^e éd. ULB, 2023.
- [2] Phillipe DEDONCKER. *Physique des télécommunications*. 2025^e éd. ULB, 2025.
- [3] EMPOSSIBLE. *Lecture 11 (FDTD) – Formulation of 2D FDTD without PML*. YouTube. 2014. URL : <https://youtu.be/f0orcF2ubIs?list=PLLYQF5WvJdJWoU9uEeWJ6-MRzDSziNnGt>.
- [4] EMPOSSIBLE. *Lecture 13 (FDTD) – The Perfectly Matched Layer*. YouTube. 2014. URL : https://youtu.be/w_NnRZlNuAA?list=PLLYQF5WvJdJWoU9uEeWJ6-MRzDSziNnGt.
- [5] EMPOSSIBLE. *Lecture 2 (FDTD) – MATLAB introduction and graphics*. YouTube. 2014. URL : <https://youtu.be/80QkkHWxHXk?list=PLLYQF5WvJdJWoU9uEeWJ6-MRzDSziNnGt>.
- [6] EMPOSSIBLE. *Lecture 3 (FDTD) – Building geometries in data arrays*. YouTube. 2014. URL : <https://youtu.be/ZUd8fuqbeCo?list=PLLYQF5WvJdJWoU9uEeWJ6-MRzDSziNnGt>.
- [7] EMPOSSIBLE. *Lecture 6 (FDTD) – Implementation of 1D FDTD*. YouTube. 2014. URL : <https://youtu.be/hNN7EtZsJuU?list=PLLYQF5WvJdJWoU9uEeWJ6-MRzDSziNnGt>.
- [8] OPENAI. *ChatGPT*. Consulté le 3 juin 2025. 2025. URL : <https://chat.openai.com>.
- [9] Frédéric ROBERT. *ELEC-H-2001 - Electricité Seconde partie : Théorie des champs*. 2018^e éd. ULB, 2018.